

Chapter 8:

Spatial Data Representation and Processing

Prof. Dr.-Ing. Thomas Schultz

URL: <http://cg.cs.uni-bonn.de/schultz/>

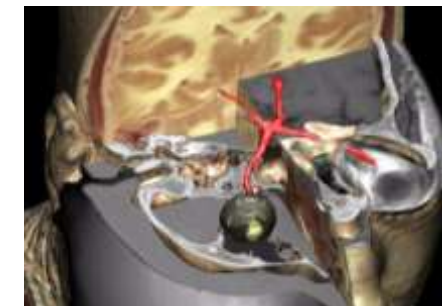
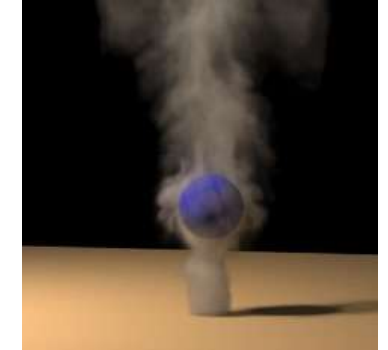
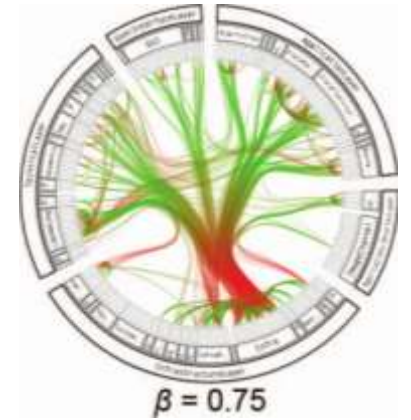
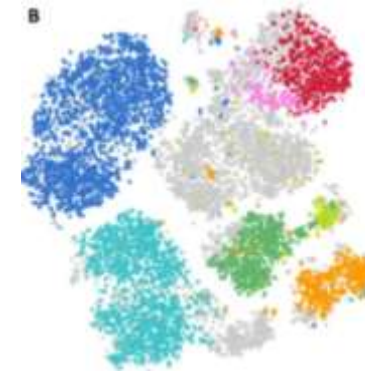
E-Mail: schultz@cs.uni-bonn.de

Office: Friedrich-Hirzebruch-Allee 6, Room 2.117

June 3/9, 2026

From InfoVis to SciVis

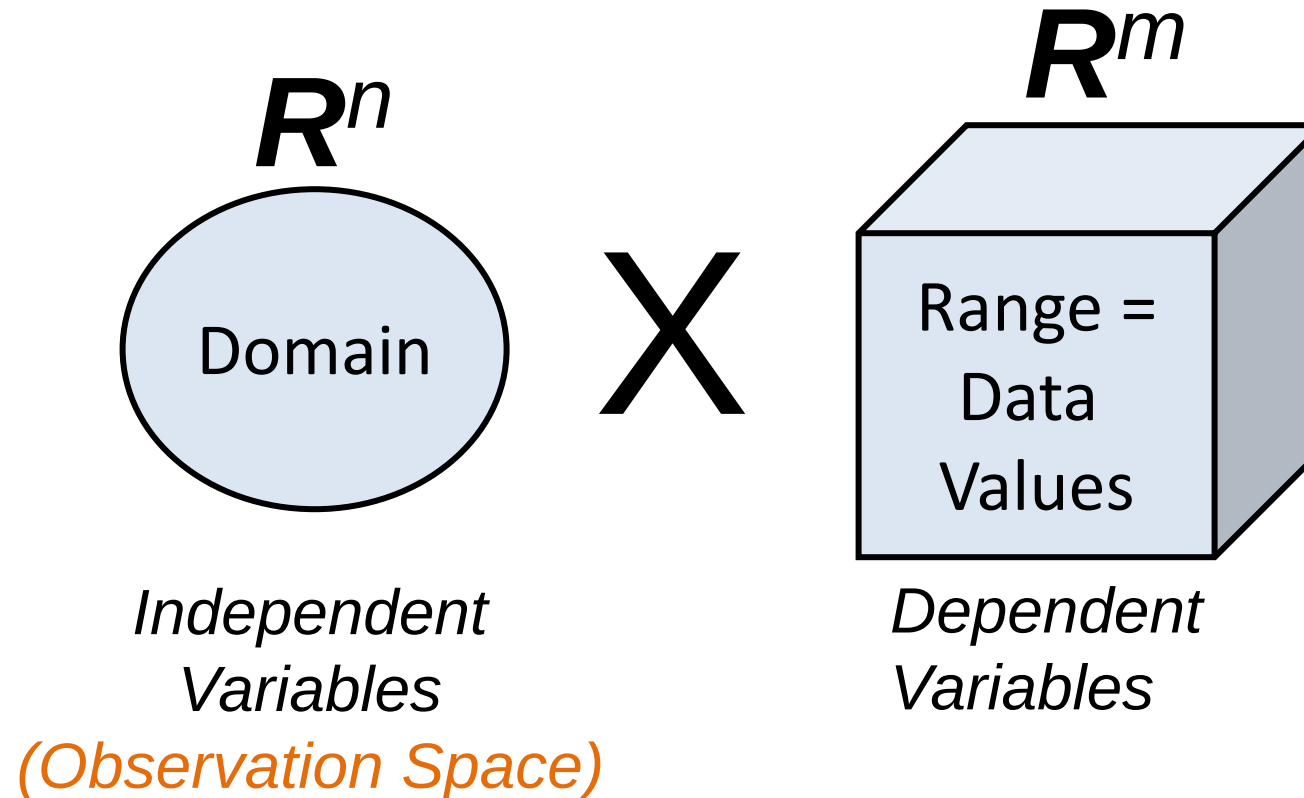
- **InfoVis:** Visual encoding and interaction with abstract data
 - *Examples:* multi- and high-dimensional data, graphs
 - GeoVis: somewhat “in between”
- **SciVis:** Visual encodings, mathematical models, algorithms, interaction techniques for intrinsically spatial data
 - Often from science, engineering, medicine



8.1 Types of Spatial Data

Representing Data as a Function

We will represent spatial data as a function $f: R^n \rightarrow R^m$



Characteristics of Domain

- **Domain** is characterized by
 - Dimension
 - Shape, determined by the positions of sample points
 - Structure (e.g., connections between samples)
 - Influence of sample points (local or global)

Characteristics of Data Values

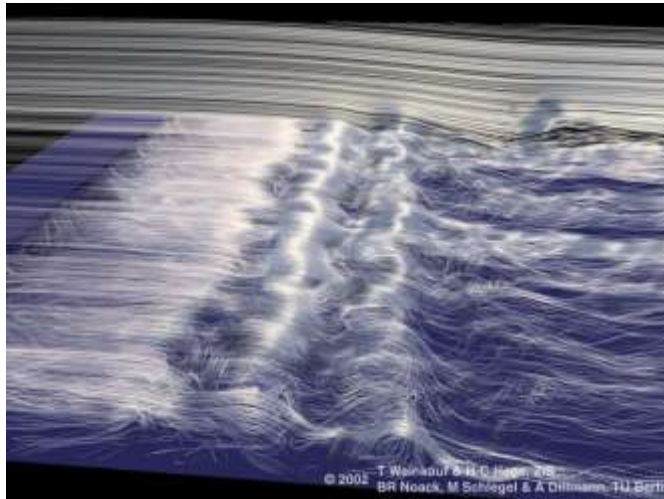
- **Data values** are characterized by their
 - Data type
 - **Scalar data** (e.g., temperature)
single value per position, $f: \mathbf{R}^n \rightarrow \mathbf{R}$
 - **Vector data** (e.g., velocity of wind)
direction and magnitude per position, $f: \mathbf{R}^n \rightarrow \mathbf{R}^m$
 - **Tensor data** (e.g., diffusion tensor)
linear map / matrix per position, $f: \mathbf{R}^n \rightarrow \mathbf{R}^{m_1 \times m_2}$
 - Dimension (number of vector/tensor components)
 - Set of possible values
 - Dynamic range: Minimum / maximum scalar value
 - Error (variance)

Classification of Visualization Techniques

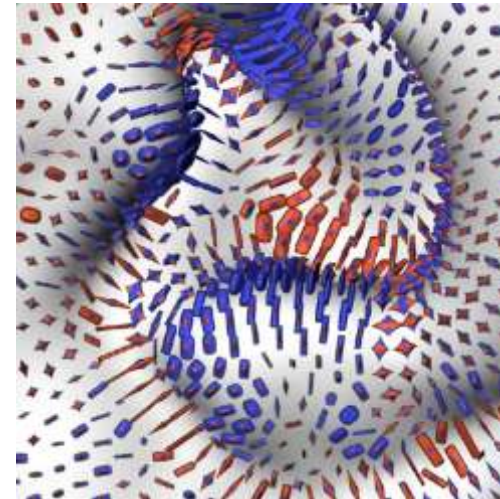
- SciVis methods classified according to
 - Dimension of the domain of the problem (independent parameters)
 - Type and dimension of the data to be visualized (dependent parameters)



3D Scalar



3D Vector



2D/3D Tensor

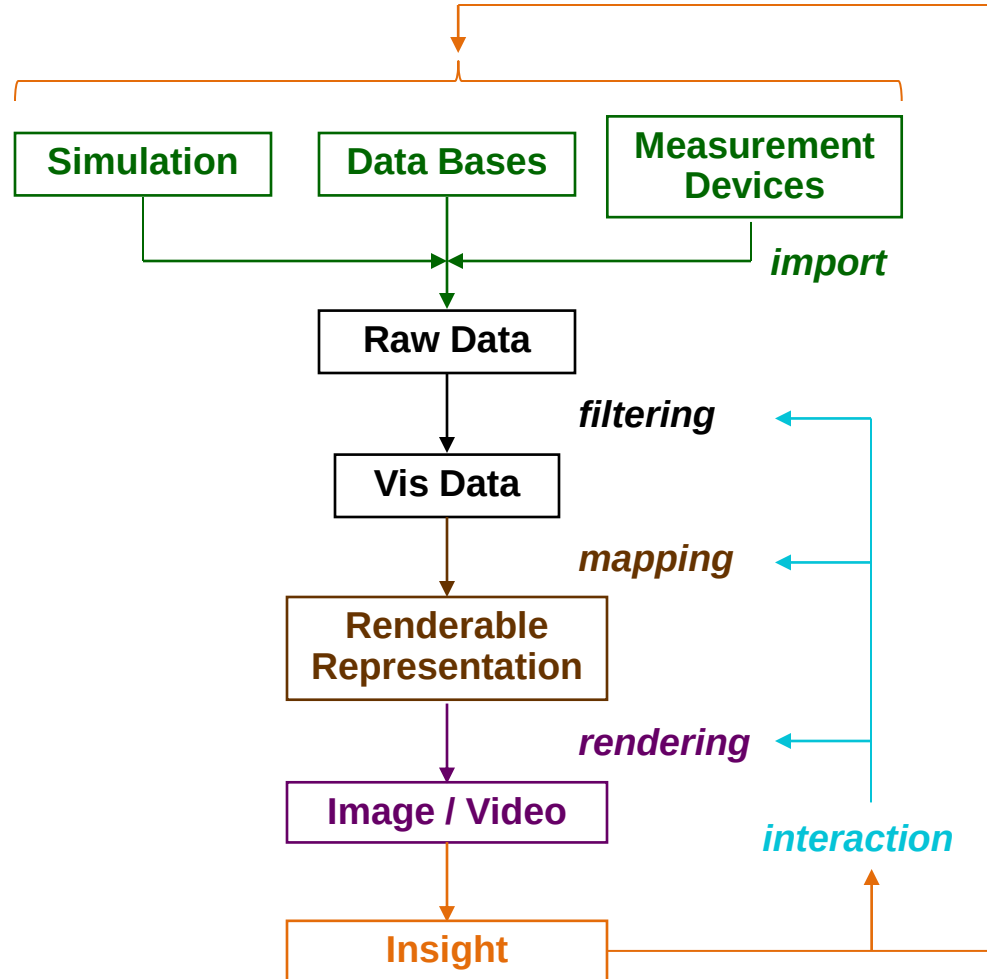
8.2 Visualization Pipelines

Visualization Pipelines: Motivation

- **Visualization Pipelines**
 - provide a scheme of how **data** is read in, filtered, mapped, and rendered during visualization
 - are a useful **abstraction** and common reference for many individually tailored solutions commonly found in visualization
 - guide organization of **class libraries** such as VTK (Visualization Toolkit)



Visualization Pipeline

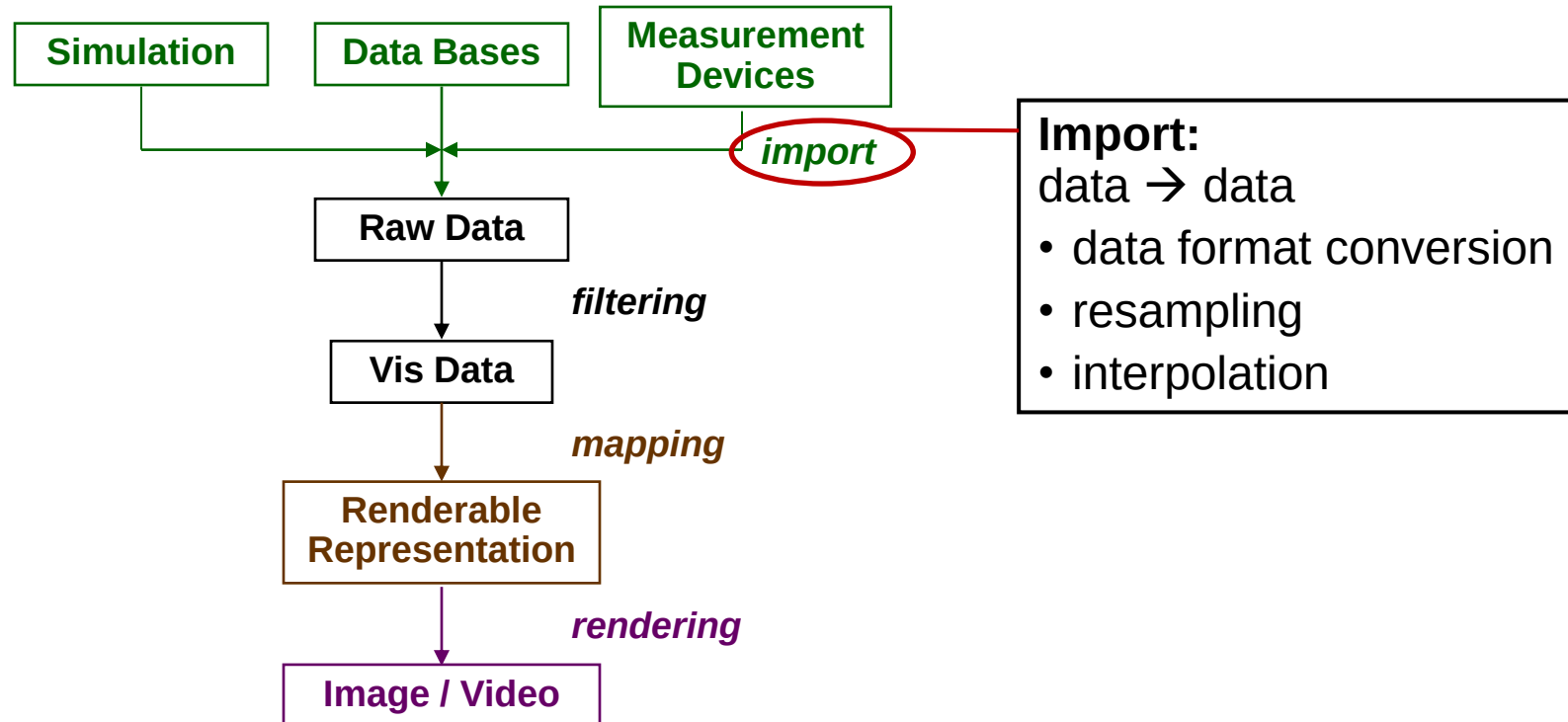


Examples of interaction:

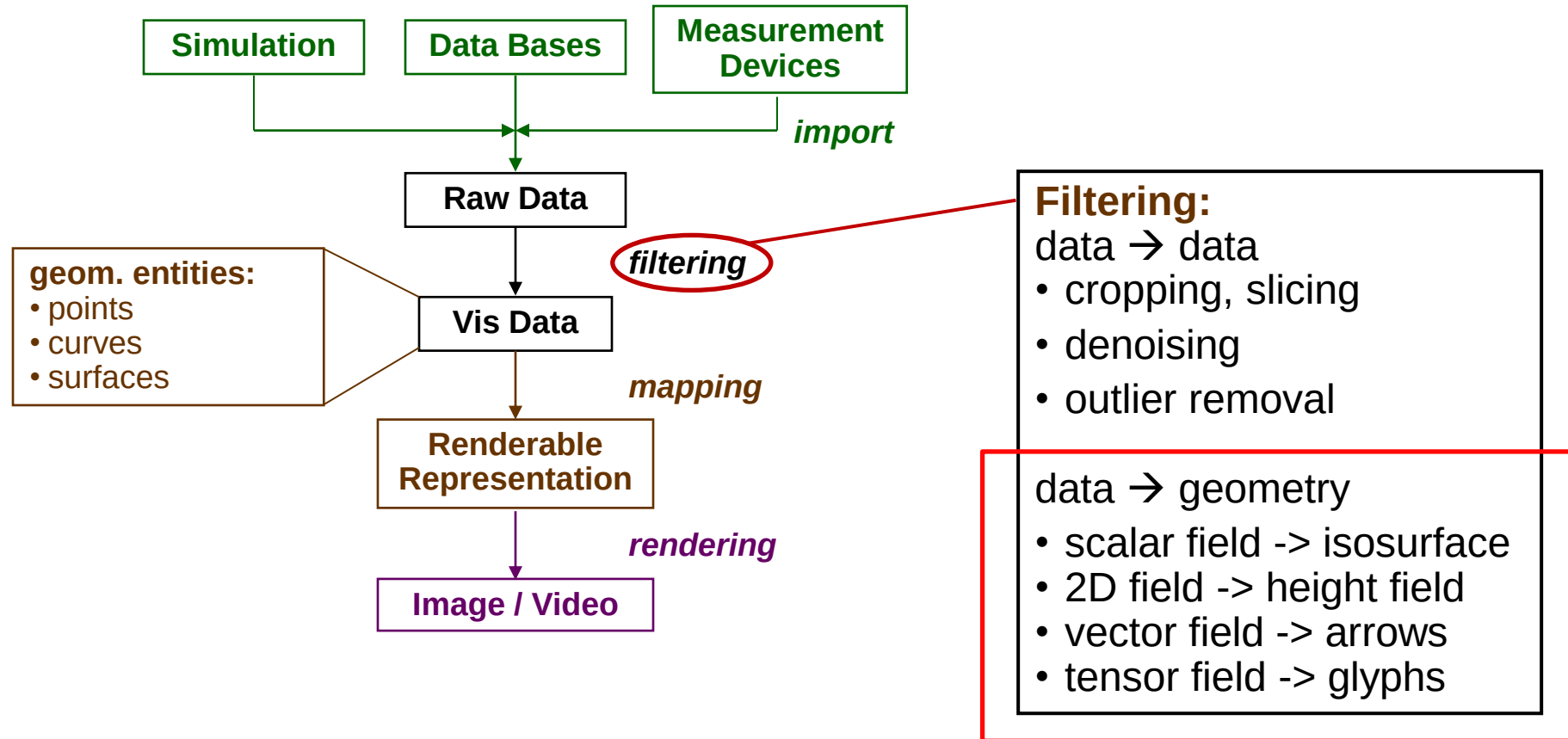
- Change viewpoint
- Filter out parts of the data
- Change level of detail
- Switch between visualization techniques

A **crucial benefit** of computer-based visualization

Import



Filtering



Counts as filtering in VTK; conceptually, is often counted as mapping

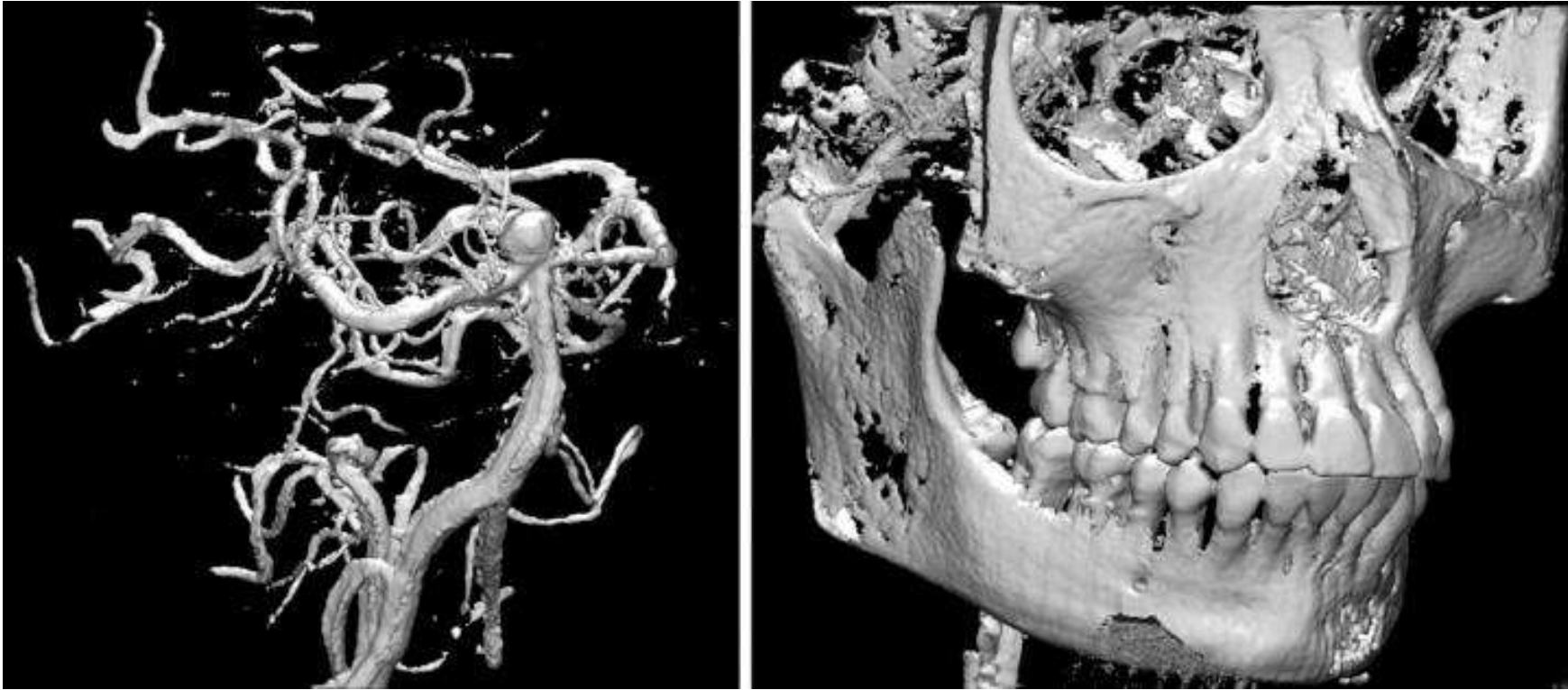
Example: Isocontours

- 2D isocontours: height lines in a topographic map

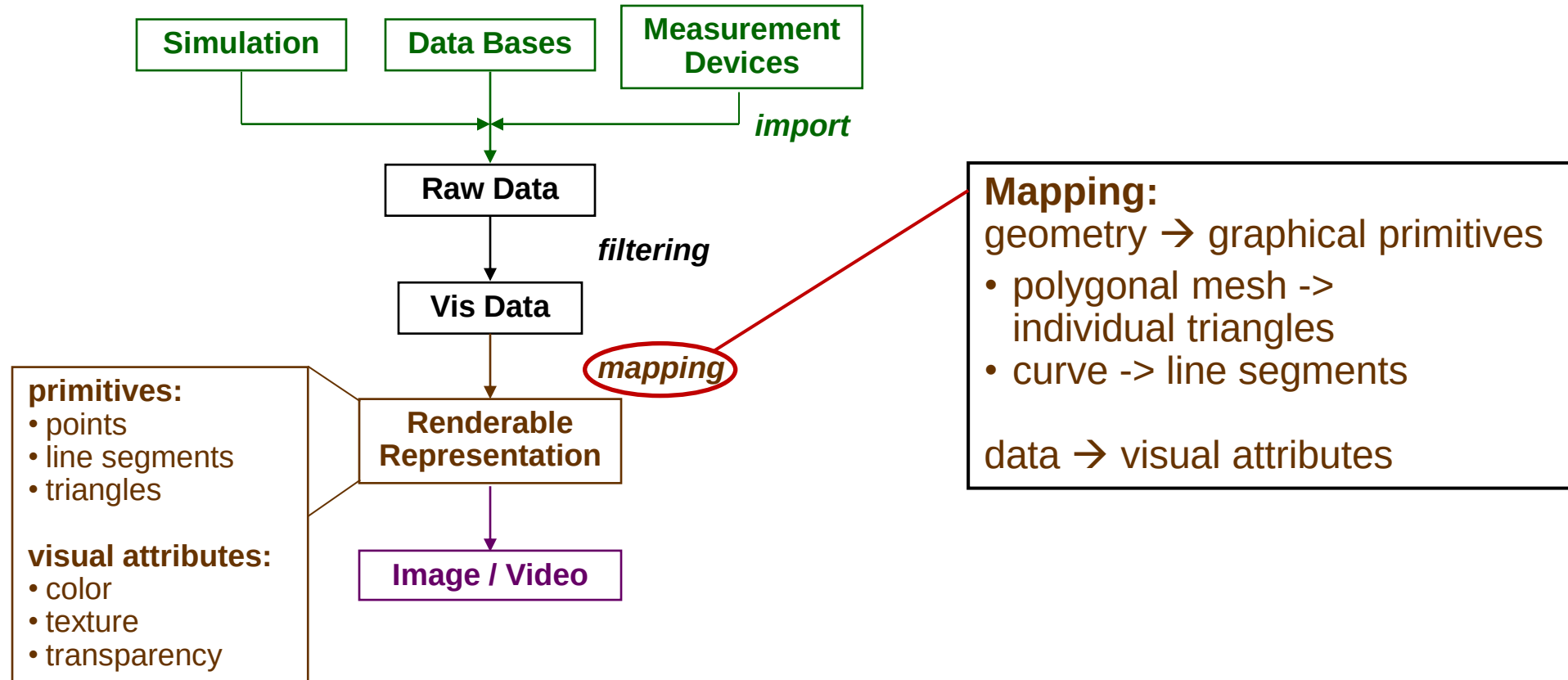


Example: Isosurfaces

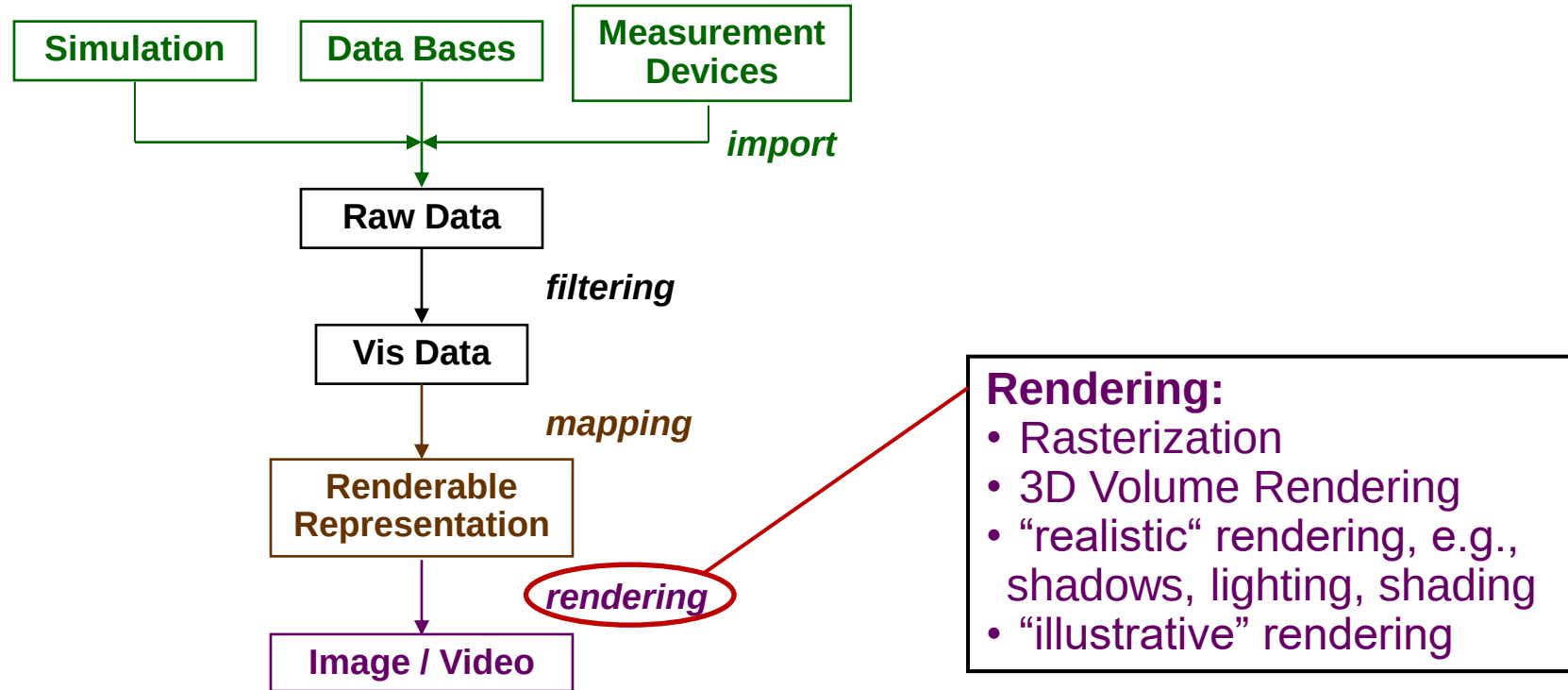
- Isosurfaces in 3D medical data often depict anatomical structures



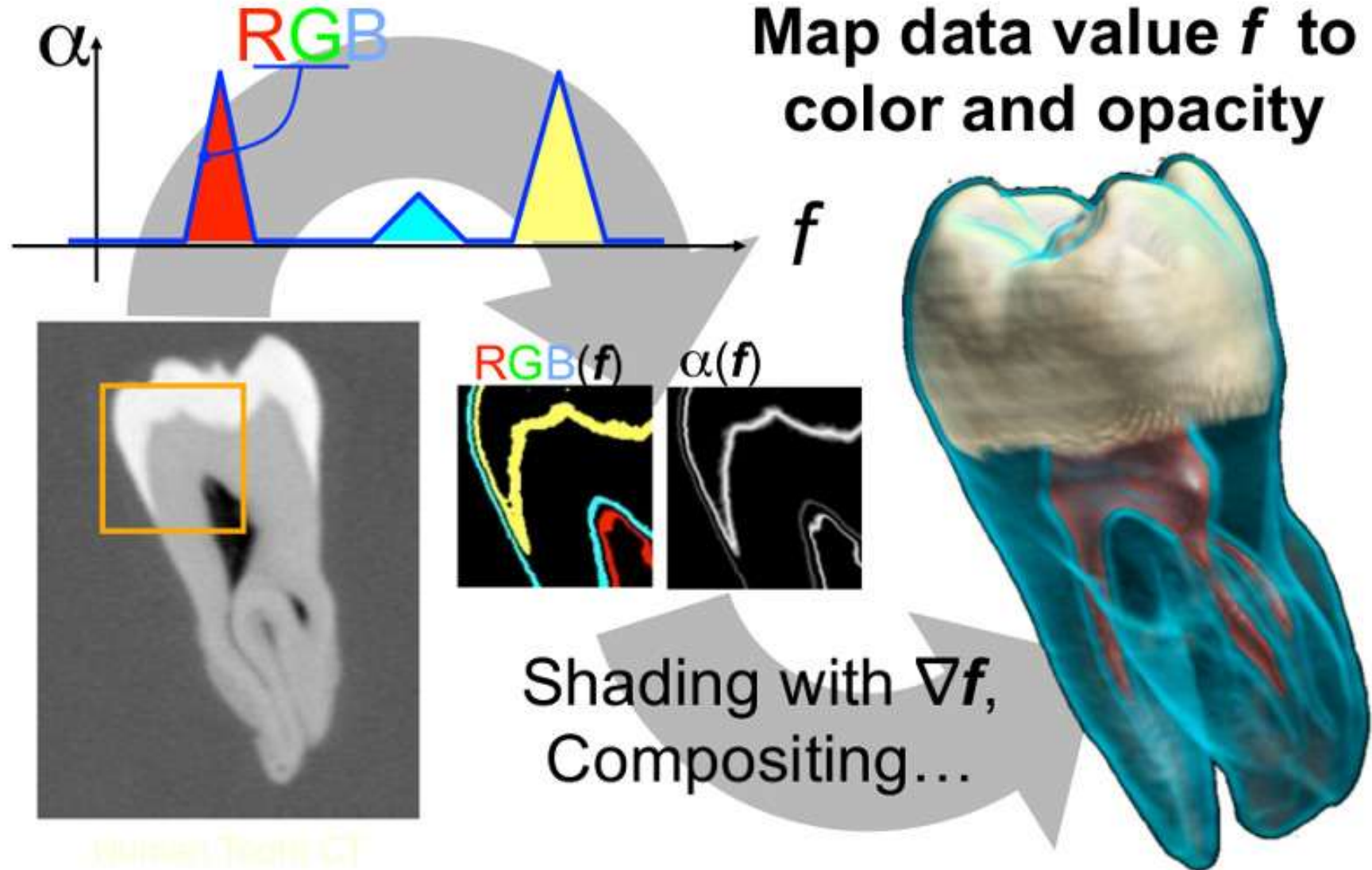
Mapping



Rendering



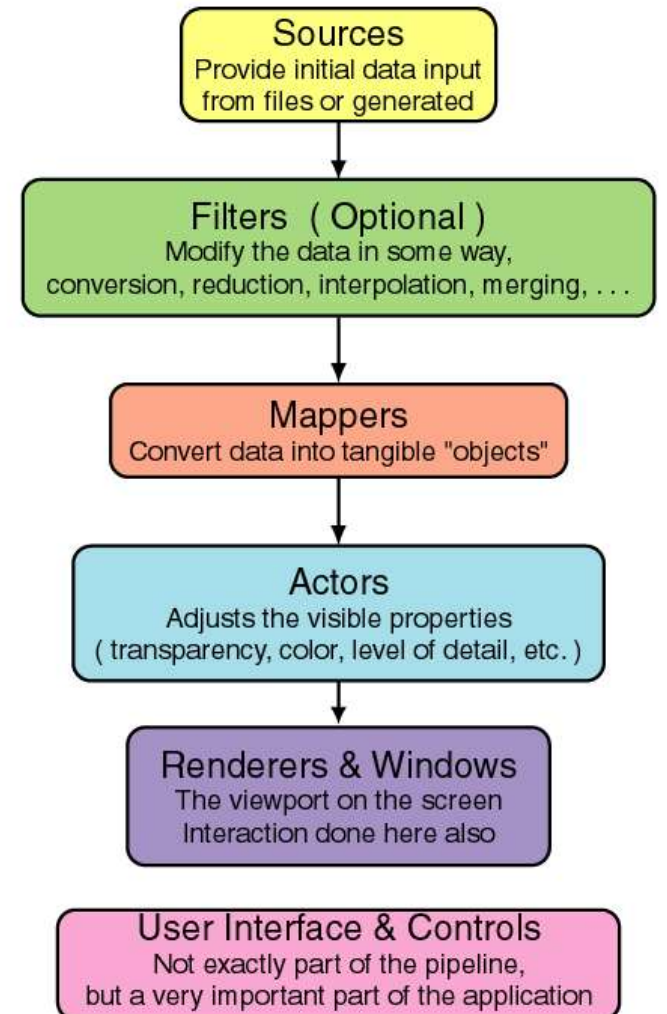
Example: Direct Volume Rendering



Visualization Pipeline in VTK

- The **Visualization Toolkit (VTK)** provides a C++ class library that implements widely used algorithms at each stage of the pipeline
 - Python bindings are available
 - Many visualization tools (e.g., paraview) use it

VTK Visualization Pipeline



Vis Pipelines: What You Should Know

- **Standard visualization pipeline** with its individual stages
 - We will return to this repeatedly for context
- How it is implemented in VTK
 - Will become relevant in assignments

8.3 Spatial Data Structures / Grids

Data Structures: Requirements

- **Considerations in choosing data structures:**
 - Convenience of access
 - Choice might depend on most frequent access patterns
 - Memory efficiency
 - Might involve lossless or lossy compression
 - Portability
 - binary – less portable, more space/time efficient
 - text – human readable, portable, less space/time efficient
- **Definitions:**
 - If points are arbitrarily distributed and no connectivity exists between them, the data is called *scattered*
 - Otherwise, the data is composed of *cells* bounded by *grid lines*
 - Topology specifies the structure (connectivity) of the data

Discrete Data Representation

- Discrete spatial data
 - Usually, the visualization data is only given at discrete locations in space and/or time
 - Discrete structures consist of samples, from which grids/meshes consisting of cells are generated
- Primitives in different dimensions

dimension	cell	mesh
0D	points	
1D	lines (edges)	polyline(–gon)
2D	triangles, quadrilaterals (rectangles)	2D mesh
3D	tetrahedra, prisms, hexahedra	3D mesh

Mesh Topology vs. Geometry

- **Geometry** of a mesh is given by the spatial positions of sample points
- **Topology** describes how samples are connected
 - remains unchanged even under distortion



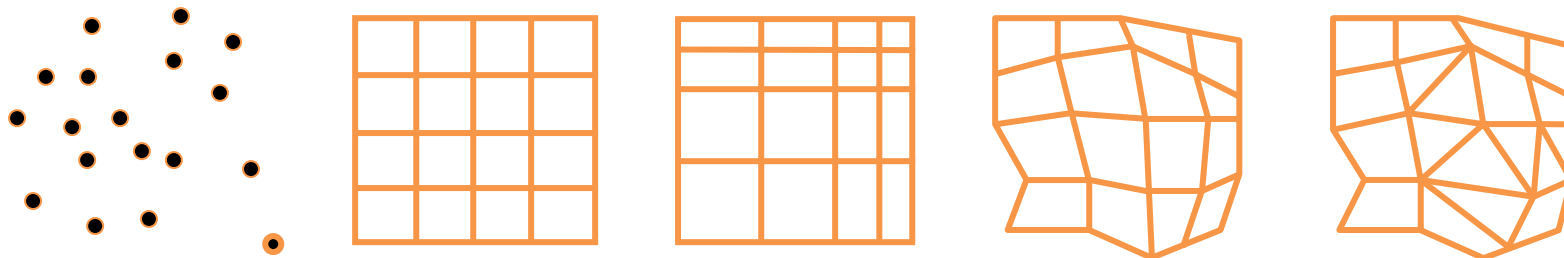
Same geometry (vertex positions)
different topology



Same topology (connections)
different geometry

Structured vs. Unstructured Grids

- In visualization, the meshes on which data is given are commonly called “grids”
- Structured and unstructured grids can be distinguished by the way the elements or cells meet
- Structured grids
 - Have a regular topology and regular / irregular geometry
- Unstructured grids
 - Have irregular topology and geometry

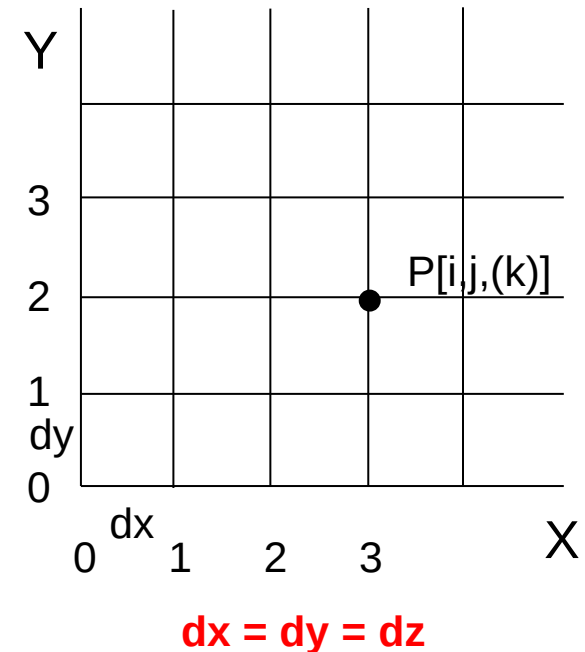


Structured Grids

- Characteristics of **structured grids**
 - Regular topology: Every interior point has the same number of neighbors
 - Often composed of sets of connected parallelograms (hexahedra), with cells being equal or distorted with respect to (non-linear) transformations
- Advantages and Disadvantages
 - + Easy to compute with
 - + Memory efficient (topology not stored explicitly)
 - May require many or badly shaped elements in order to cover the underlying domain

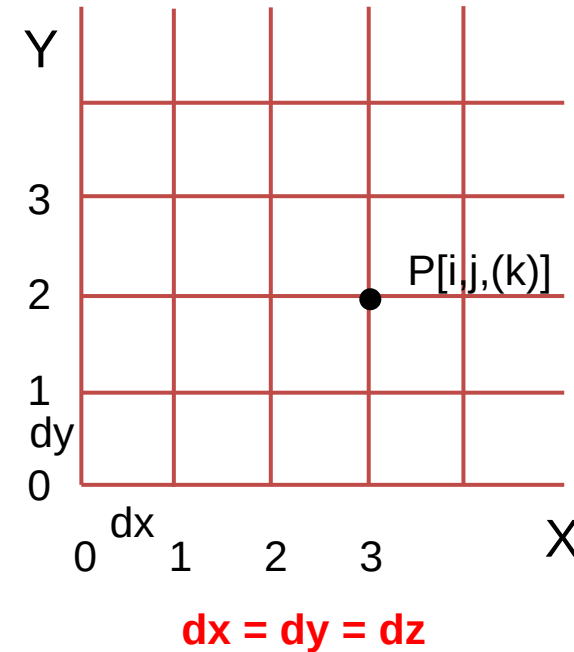
Cartesian Grids

- Cartesian grids
 - Structured grid with orthogonal axes and equidistant grid points
 - Cells and points are numbered sequentially with respect to increasing X, then Y, then Z, or vice versa
 - Number of points = $N_x \bullet N_y \bullet N_z$
 - Number of cells = $(N_x - 1) \bullet (N_y - 1) \bullet (N_z - 1)$



Cartesian Grids: Memory Consumption

- Cartesian grids
 - Vertex positions are given implicitly from $[i,j,k]$:
 - $P[i,j,k].x = i \cdot dx$
 - $P[i,j,k].y = j \cdot dy$
 - $P[i,j,k].z = k \cdot dz$
 - Memory consumption:
 - $N_x \cdot N_y \cdot N_z$ data values
 - 1 integer for dimension D
 - D integers to store axis lengths
 - 1 float to store $dx(=dy=dz)$
 - Optional (see Section 8.6):
 - D floats to store origin
 - D rotation angles for axes

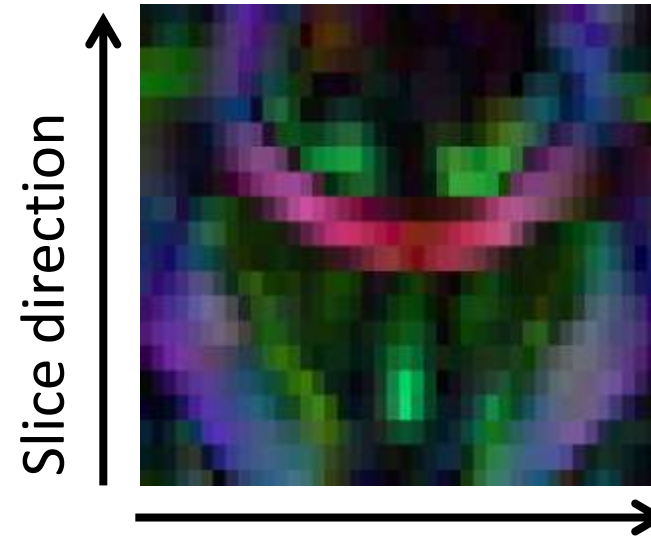


Voxel Anisotropy

- The voxels in a Cartesian grid are called *isotropic*.
- Many 3D acquisition techniques measure a series of 2D slices that are thicker than the in-plane resolution. This leads to uniform grids with *anisotropic voxels*:



In-plane slice



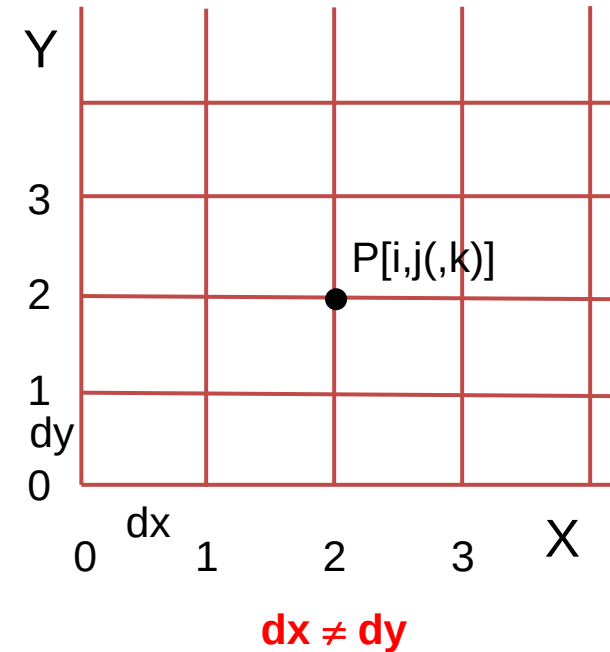
In-plane direction

Uniform Grids

- Uniform grids generalize Cartesian grids:
 - Spacing between grid points is constant in each dimension
-> same indexing scheme as for Cartesian grids
 - But resolution / spacing can be different in each dimension,
 $dx \neq dy (\neq dz)$
 - In rare cases, axes may be non-orthogonal

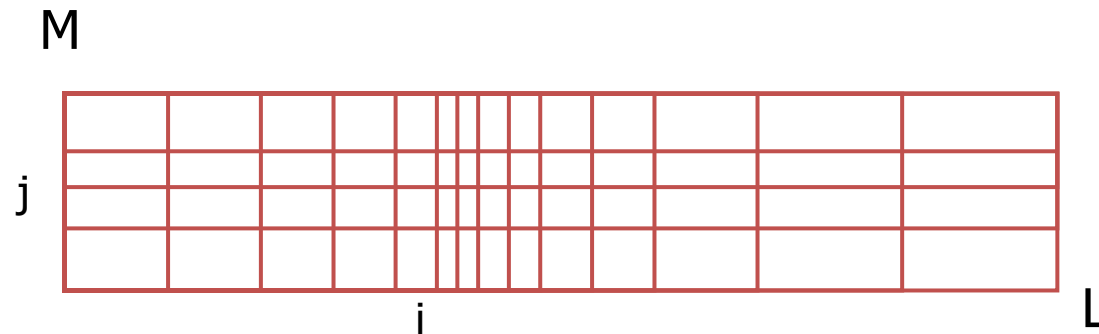
Uniform Grids: Memory Consumption

- Uniform Grids = “raster data”
 - Most frequent grid type in SciVis part of our lecture
- Memory consumption:
 - $N_x \cdot N_y \cdot N_z$ data values
 - 1 integer for dimension D
 - D integers to store axis lengths
 - D floats for distances
 - Optional:
 - D floats to store origin
 - $D \cdot D$ floats to store axis vectors



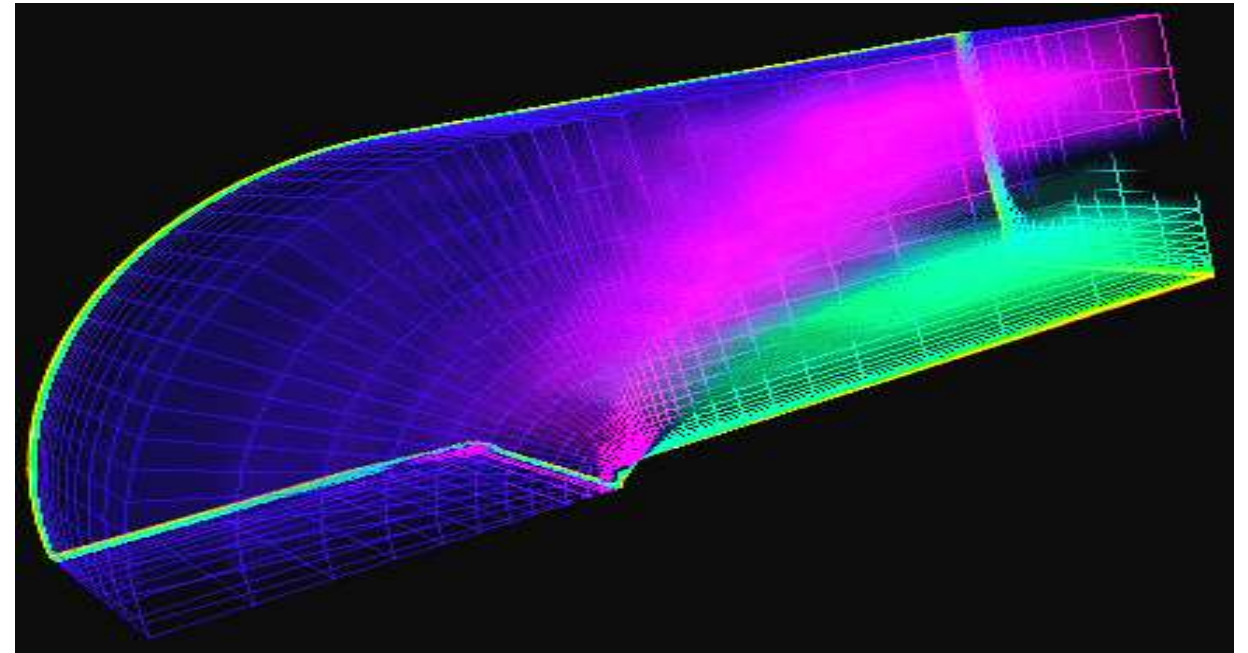
Rectilinear Grids

- Rectilinear grids
 - Topology is still regular and axis directions are fixed, but irregular spacing between grid points
 - Non-linear scaling of positions along either axis
 - Spacing must be stored explicitly:
 $x_coord[L-1]$, $y_coord[M-1]$, $z_coord[N-1]$ (1D arrays)
 - Topology is still implicit



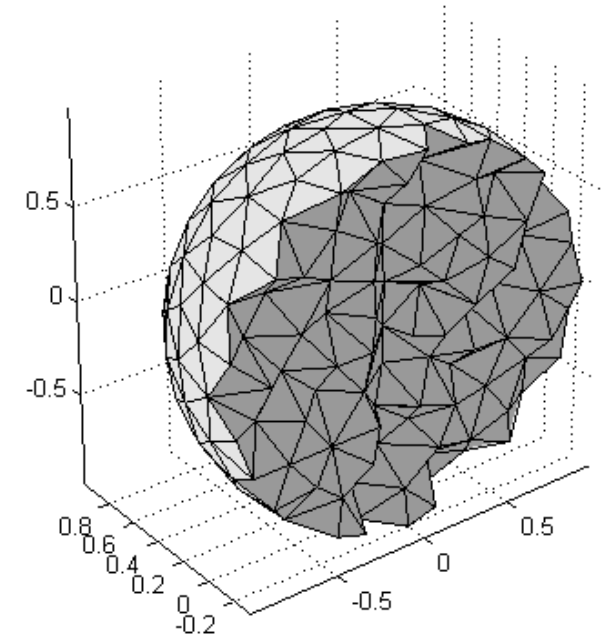
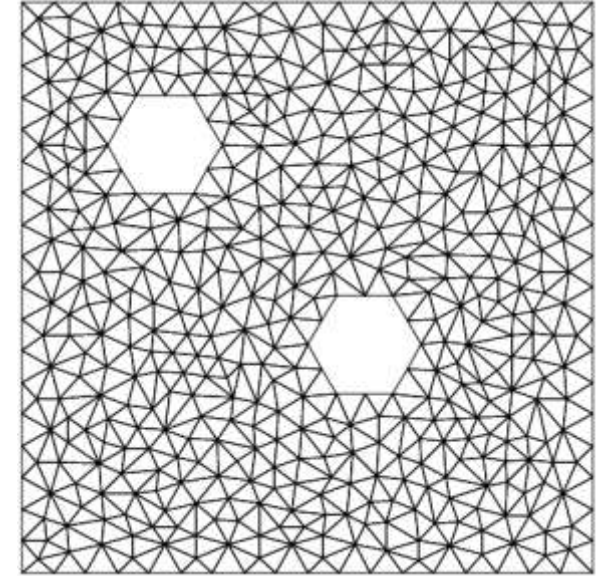
Curvilinear Grids

- Curvilinear grids
 - Topology is still regular but vertex positions are non-linearly transformed
 - Need to explicitly store all positions:
 - $x_coord[L,M,N]$
 - $y_coord[L,M,N]$
 - $z_coord[L,M,N]$
 - Topology is still implicit
 - Geometric structure might result in concave grids



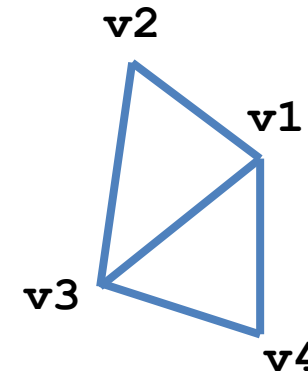
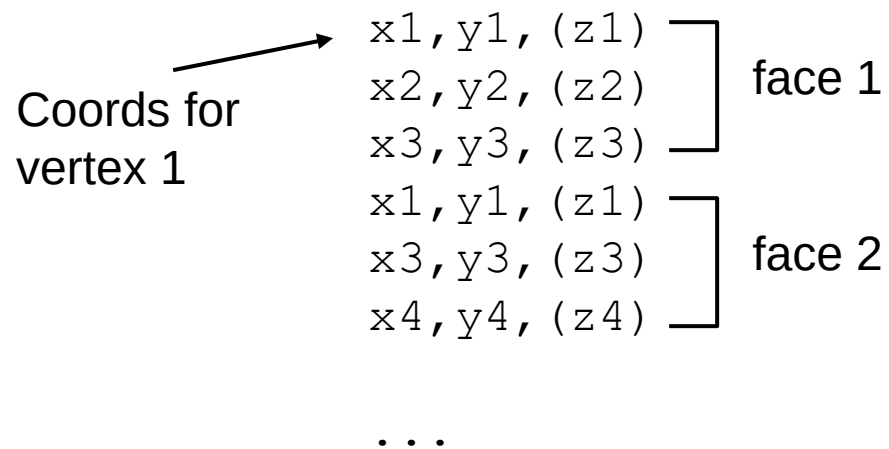
Unstructured Grids

- **Unstructured grids** have irregular topology
 - Might result from triangulation of (scattered) point sets
 - Might consist of one unique element type or they can be hybrid
 - Can also arise as quadtrees (in 2D) or octrees (in 3D)
- Characteristics of unstructured grids
 - + Usually fewer elements are needed to cover the domain
 - Grid points and connectivity must be stored
 - Specific data structures needed to allow for efficient traversal and thus data retrieval



Implementing Unstructured Grids

- Simplest implementations of unstructured grids:
 - Direct form



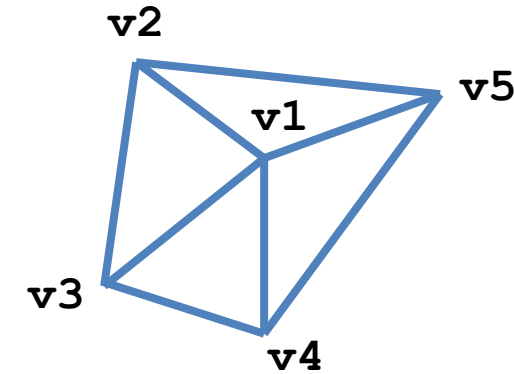
- Additionally store the data values
- Problems: storage space, redundancy

Indexed Face Sets

- More memory efficient implementation:

- Indexed face set

	vertex list	face list
Coords for vertex 1	x1, y1, (z1)	1, 2, 3
	x2, y2, (z2)	1, 3, 4
	x3, y3, (z3)	1, 4, 5
	x4, y4, (z4)	1, 5, 2
	x5, y5, (z5)	...
	...	



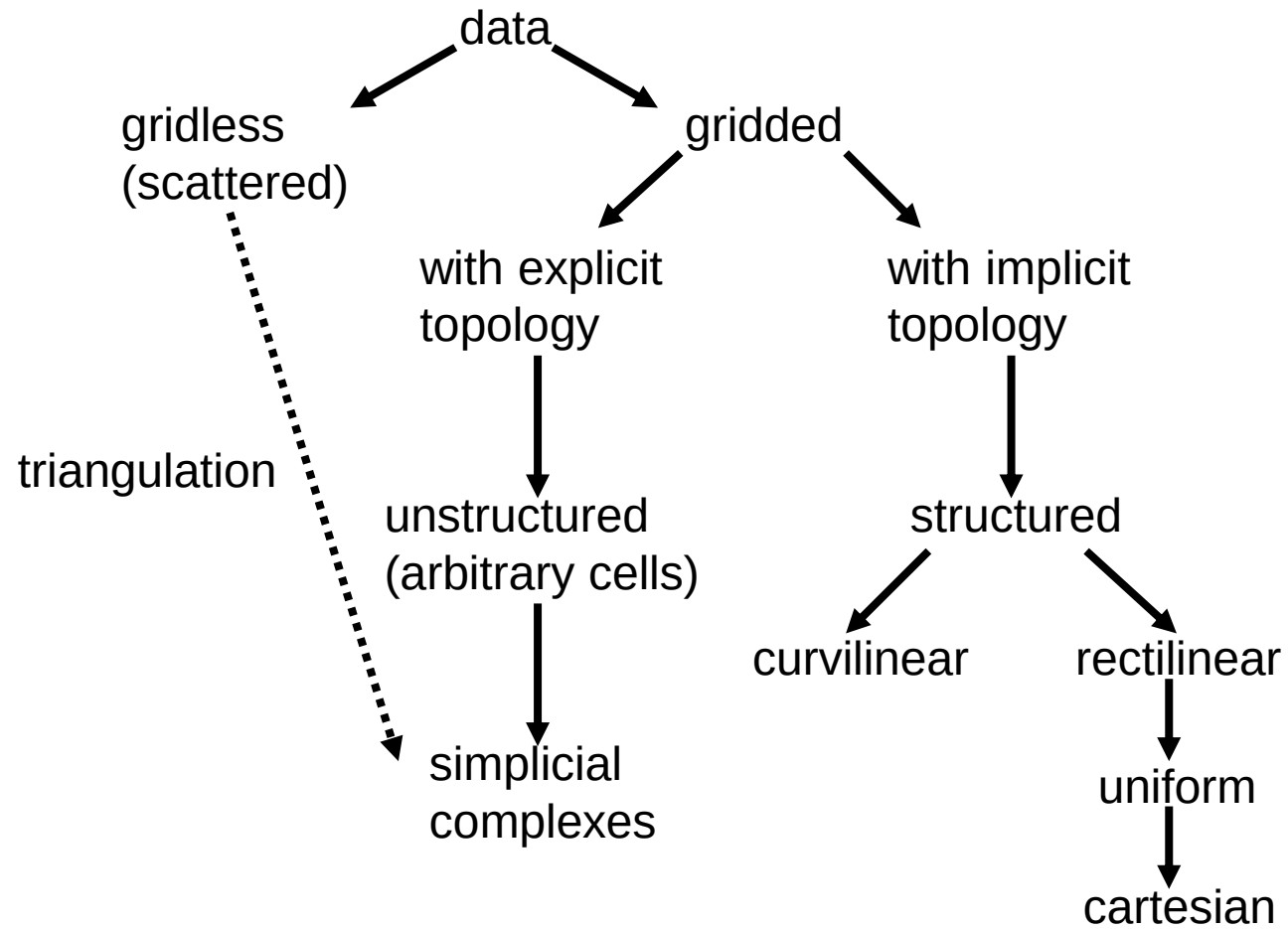
- Widely used in visualization

- e.g., rendering triangle meshes in OpenGL

- More sophisticated data structures (winged edge, half-edge...) exist to efficiently find adjacent vertices

- Attend a Computer Graphics lecture to learn about details

Overview of Data Structures



VTK File Formats

- In the exercises, we will primarily deal with the VTK file formats
 - Three types of formats, read / written with different classes
 - https://docs.vtk.org/en/latest/vtk_file_formats/index.html
- Classic (legacy) .vtk format
 - ASCII header + data
 - Designed to be easy to read and write
 - Supported dataset types:
 - Structured points (Uniform grid)
 - Rectilinear grid
 - Structured grid (Curvilinear grid)
 - Polygonal data (polygons, points, lines)
 - Unstructured grid
 - Each types requires its own class, e.g., vtkStructuredPointsReader

XML-based VTK File Formats

- More modern format, compatible with XML standard
 - More detailed meta data can be stored
 - Will be used in the exercises
 - Requires vtkXML*Reader classes / modules
- Different file extensions:
 - .vti — ImageData (uniform grid)
 - .vtr — RectilinearGrid
 - .vts — StructuredGrid (curvilinear)
 - .vtp — PolyData (unstructured)
 - .vtu — UnstructuredGrid

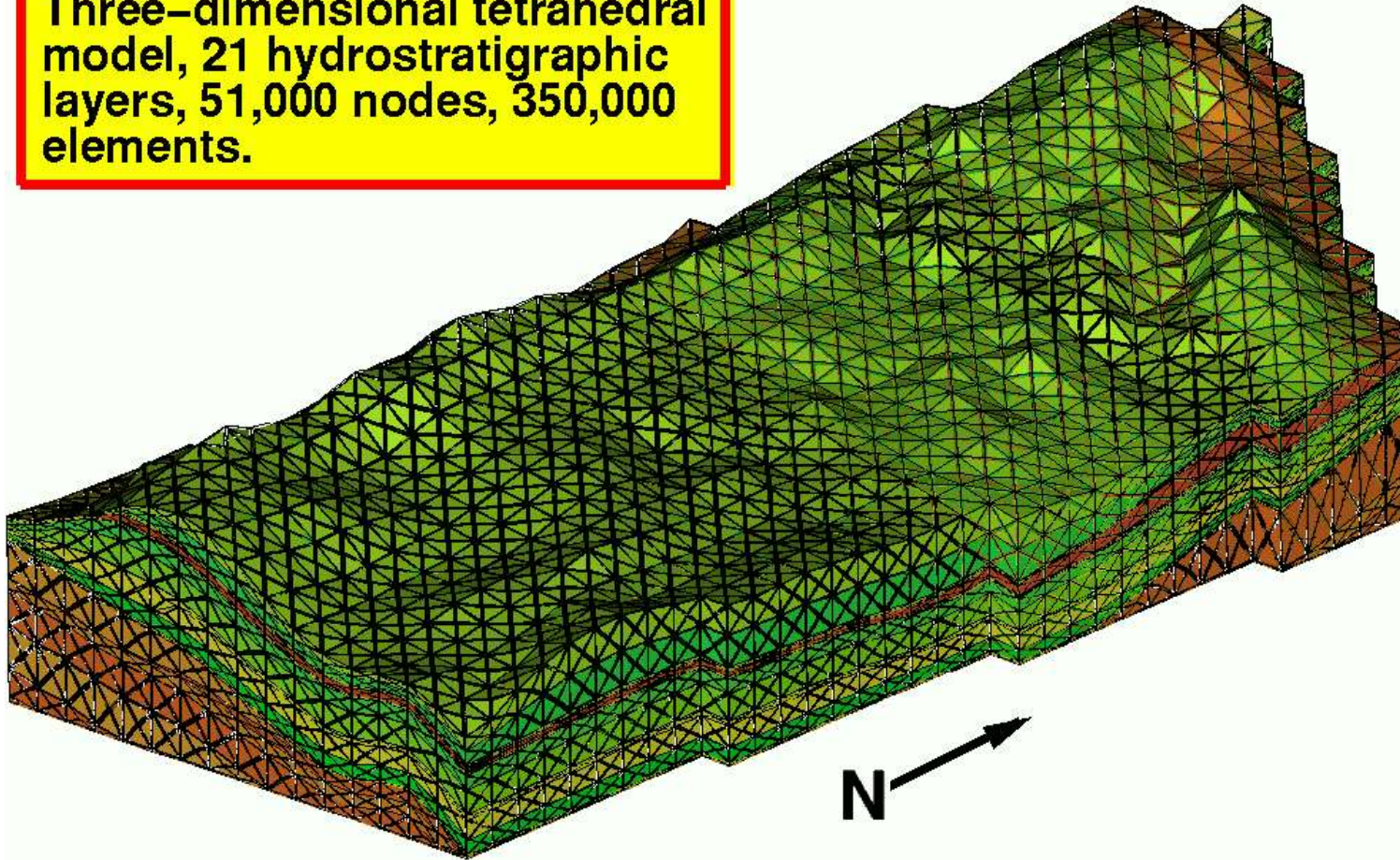
HDF-based VTK File Format

- VTKHDF is the most recent format (extension: .vtkhdf)
 - Most important dataset types are supported, including ImageData, PolyData, UnstructuredGrid
 - Same meta data can be stored as in XML-based format
 - Requires vtkHDFReader class
- Based on Hierarchical Data Format (HDF5)
 - Open source file format for large, complex, heterogeneous data
 - Optimized I/O efficiency

Example Grids

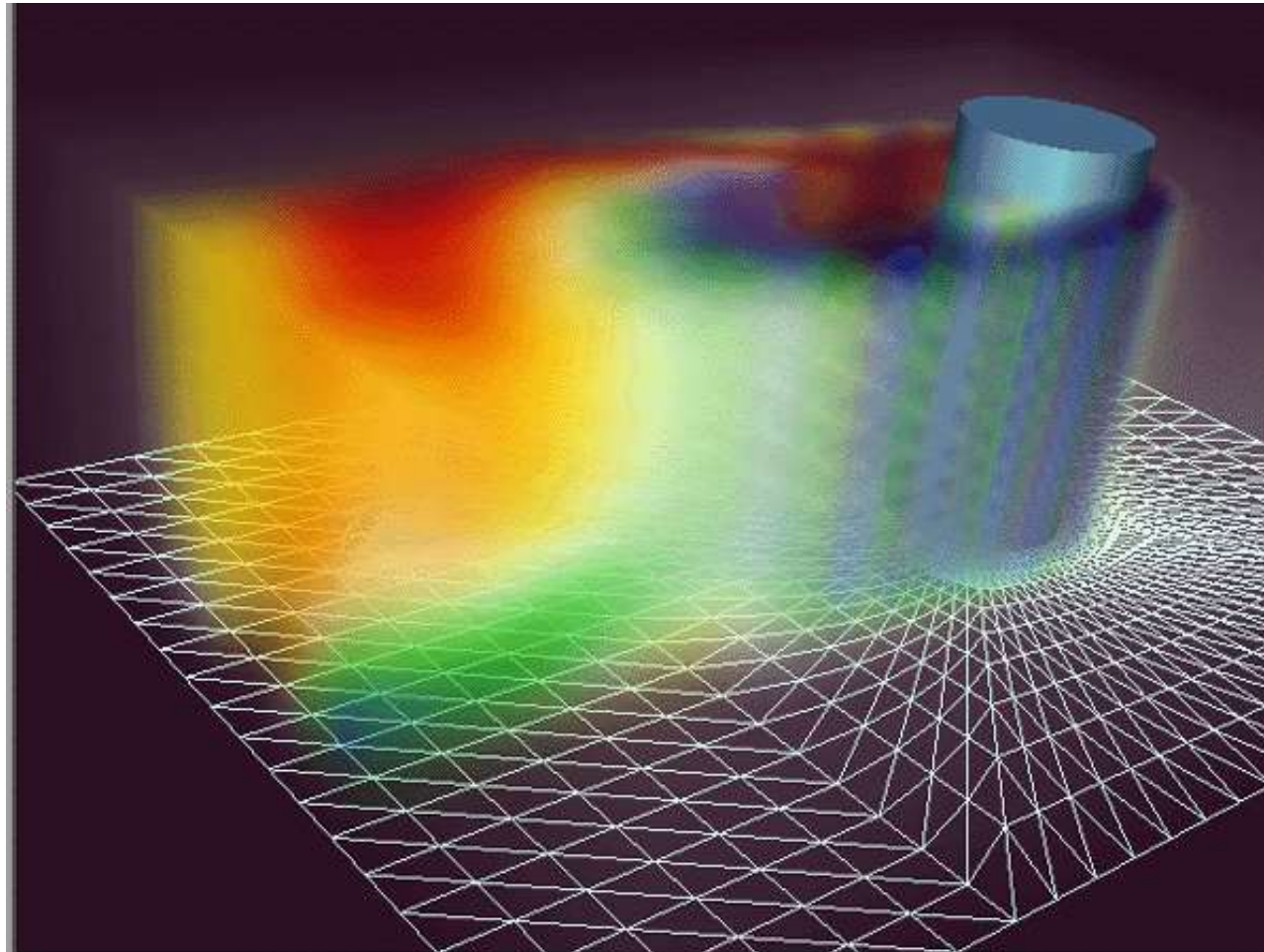
- Curvilinear grid

Three-dimensional tetrahedral model, 21 hydrostratigraphic layers, 51,000 nodes, 350,000 elements.



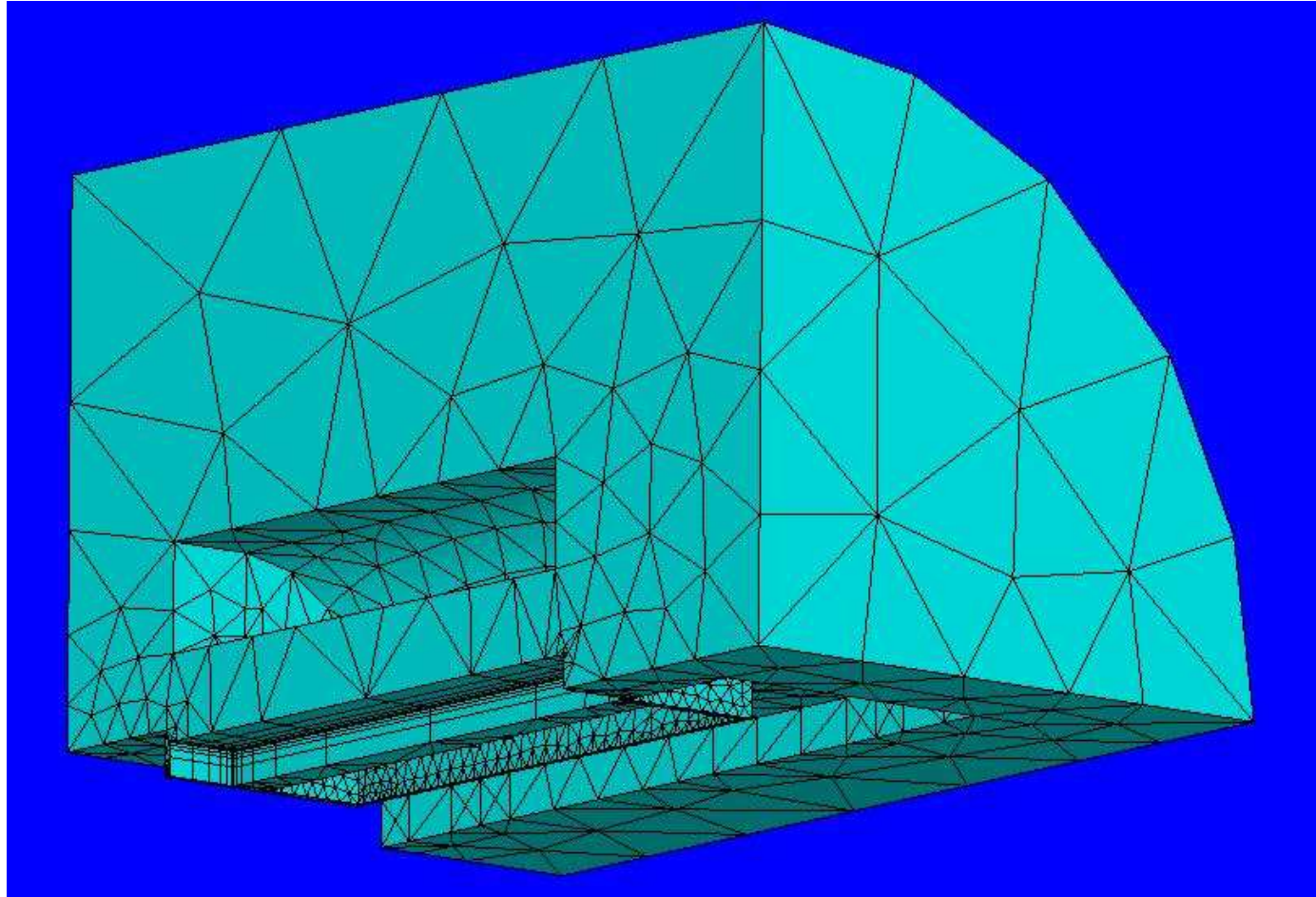
Example Grids

- Hybrid (Curvilinear / Uniform)



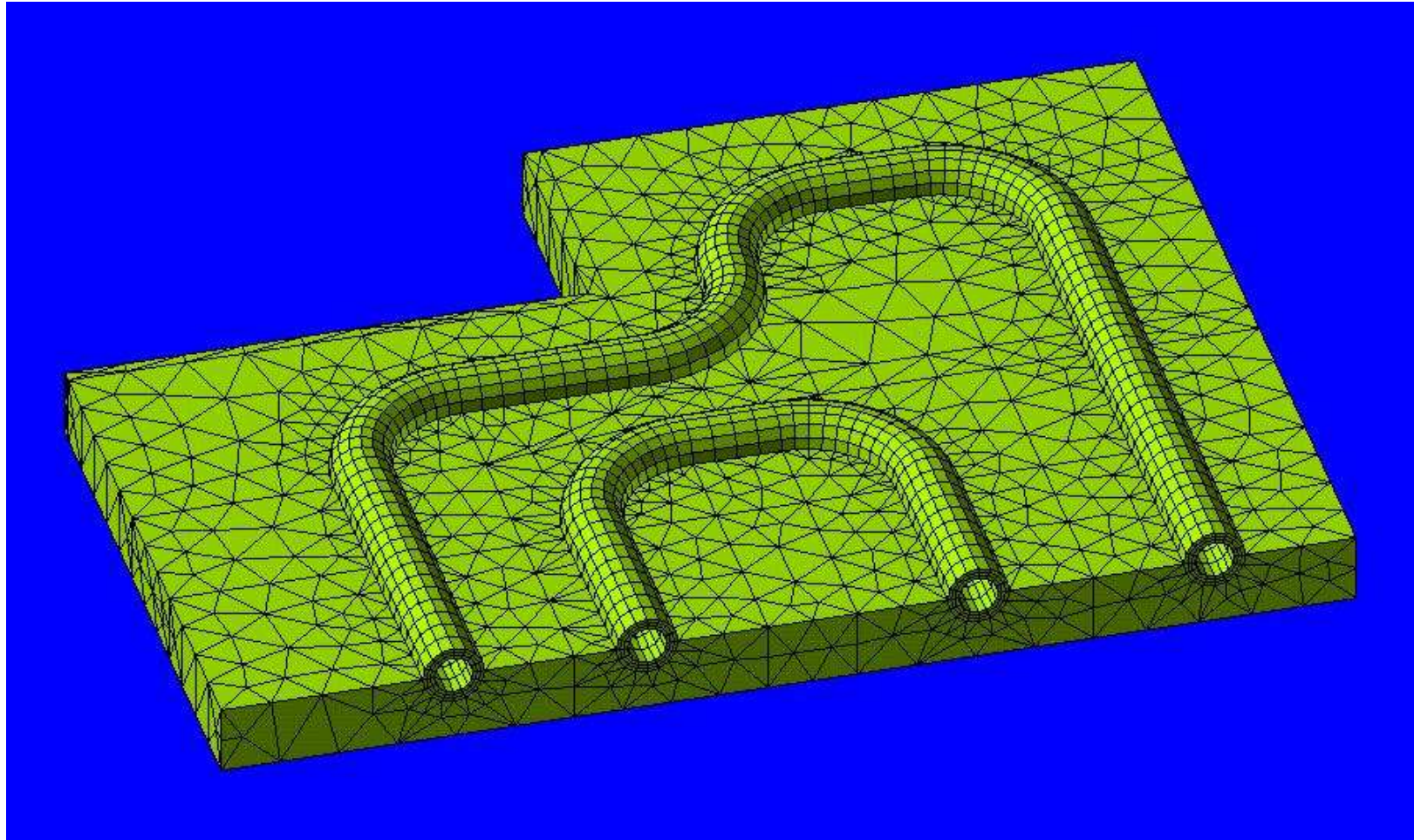
Example Grids

- Unstructured Grid (Triangle Mesh)



Example Grids

- Hybrid (Curvilinear / Unstructured)



8.4 Spatial Data Interpolation

Goal of Interpolation

- Interpolation is used to reconstruct a function $f(\mathbf{x})$ on $\mathbf{x} \in \mathbb{R}^n$ from discrete samples $\mathbf{x}_i$
 - Often, want $f(\mathbf{x})$ to be continuous or even smooth
- We already know about scattered data interpolation and linear interpolation (Chapter 6.4)
- We will now cover
 - Bi-/trilinear interpolation on rectangular/hexahedral cells
 - Smooth reconstructions via convolutions

Bilinear Interpolation On Rectangles

- **Bilinear interpolation:** To get $f(x,y)$
 - First, interpolate linearly to the correct x along the top and bottom edges

$$f_j = (1-\alpha)f_{i,j} + \alpha f_{i+1,j}$$

$$f_{j+1} = (1-\alpha)f_{i,j+1} + \alpha f_{i+1,j+1}$$

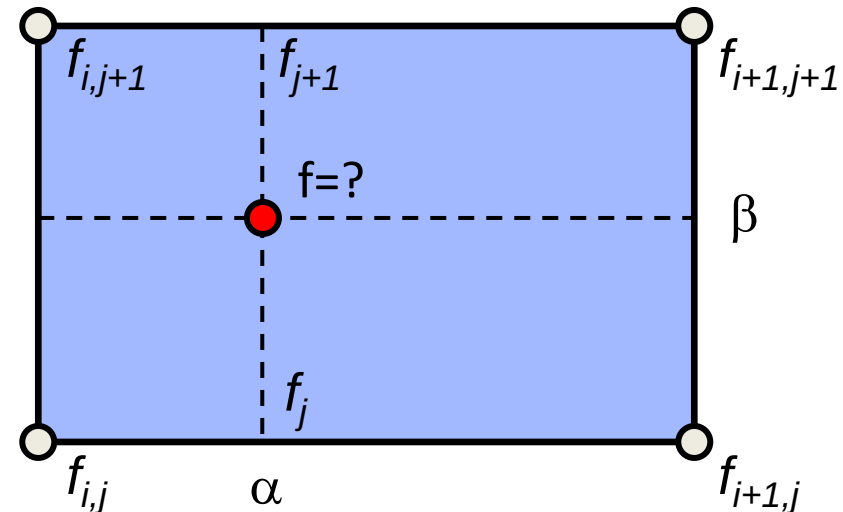
- Then, interpolate again linearly between intermediate results

$$f(x,y) = (1-\beta) f_j + \beta f_{j+1}$$

$$\alpha = \frac{x - x_i}{x_{i+1} - x_i},$$

$$\beta = \frac{y - y_i}{y_{i+1} - y_i},$$

$$\alpha, \beta \in [0,1]$$

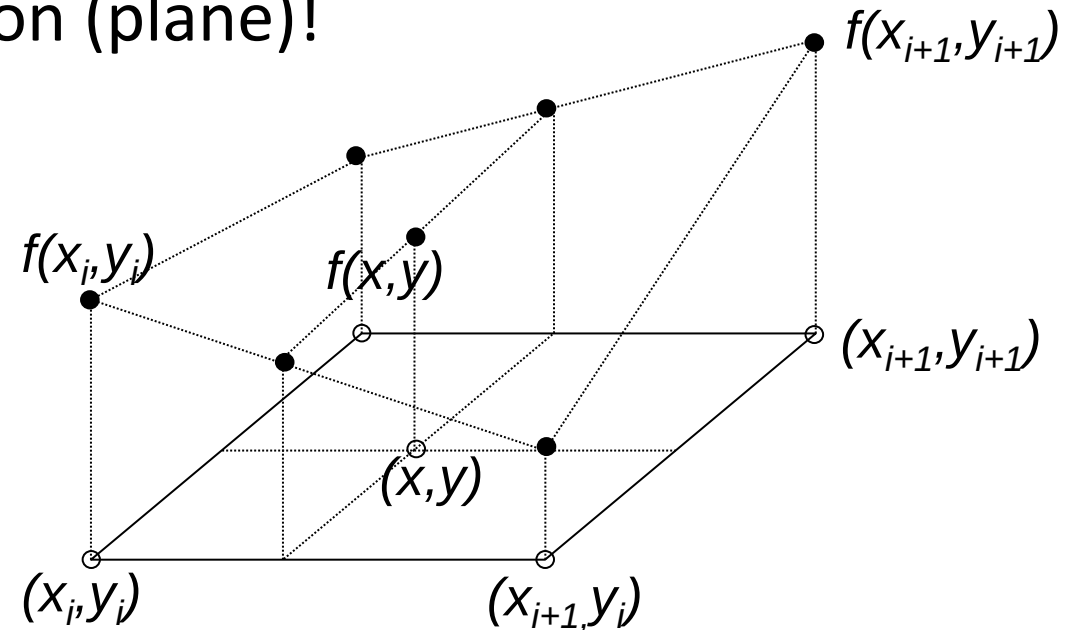


Bilinear Interpolation: Order?

- Substituting f_j and f_{j+1} and multiplying out:

$$f(x, y) = (1 - \alpha)(1 - \beta)f_{i,j} + \alpha(1 - \beta)f_{i+1,j} \\ + (1 - \alpha)\beta f_{i,j+1} + \alpha\beta f_{i+1,j+1}$$

- Shows that order of x, y does not matter
- Result is *not* a linear function (plane)!

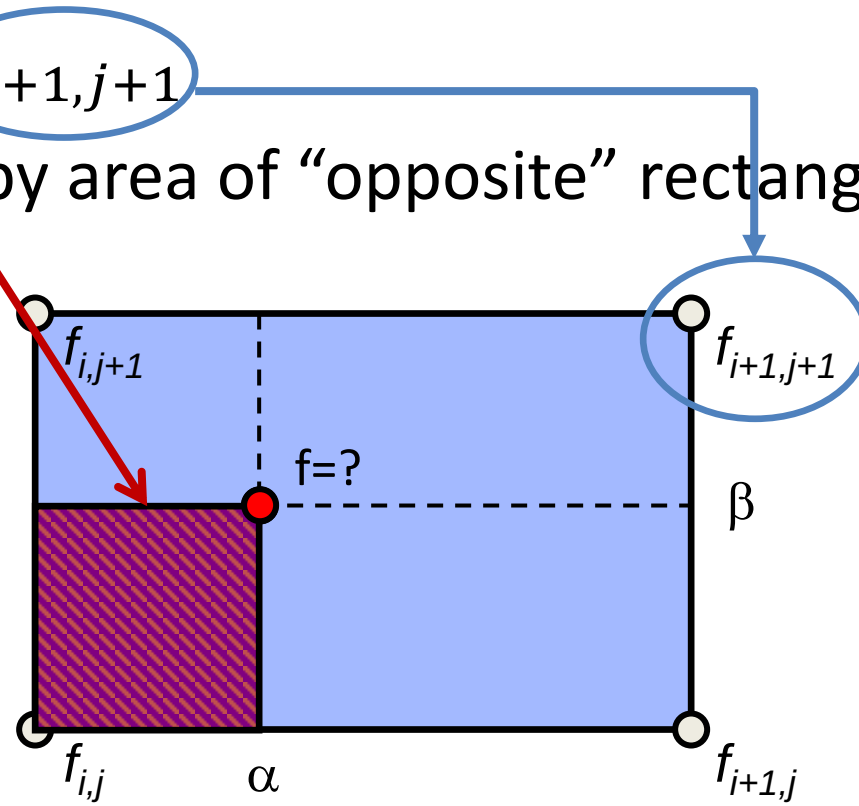


Geometric Interpretation

- Geometric interpretation of equation

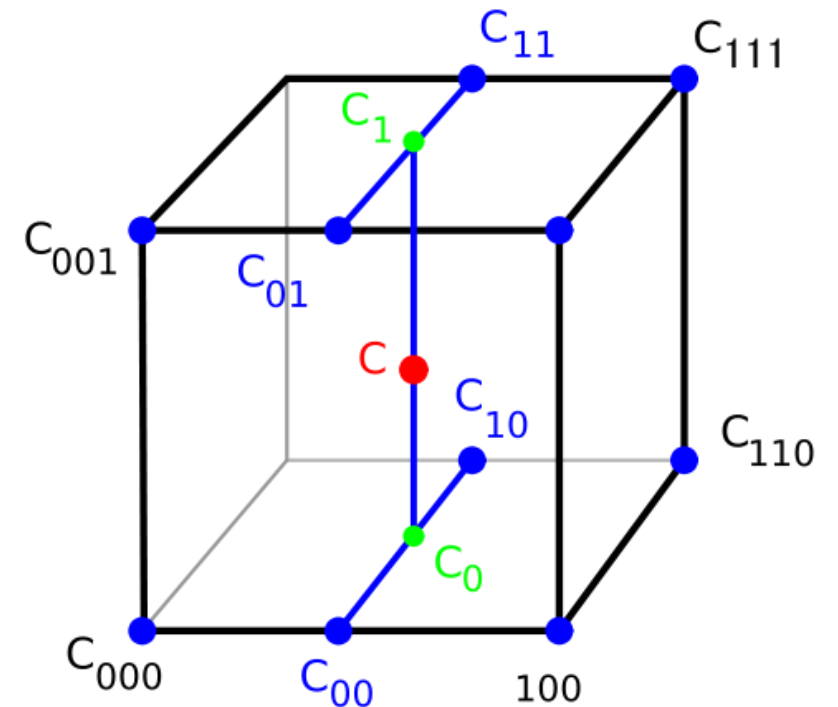
$$f(x, y) = (1 - \alpha)(1 - \beta)f_{i,j} + \alpha(1 - \beta)f_{i+1,j} \\ + (1 - \alpha)\beta f_{i,j+1} + \alpha\beta f_{i+1,j+1}$$

- Value at each corner is weighted by area of “opposite” rectangle
- Similarity to barycentric interpolation



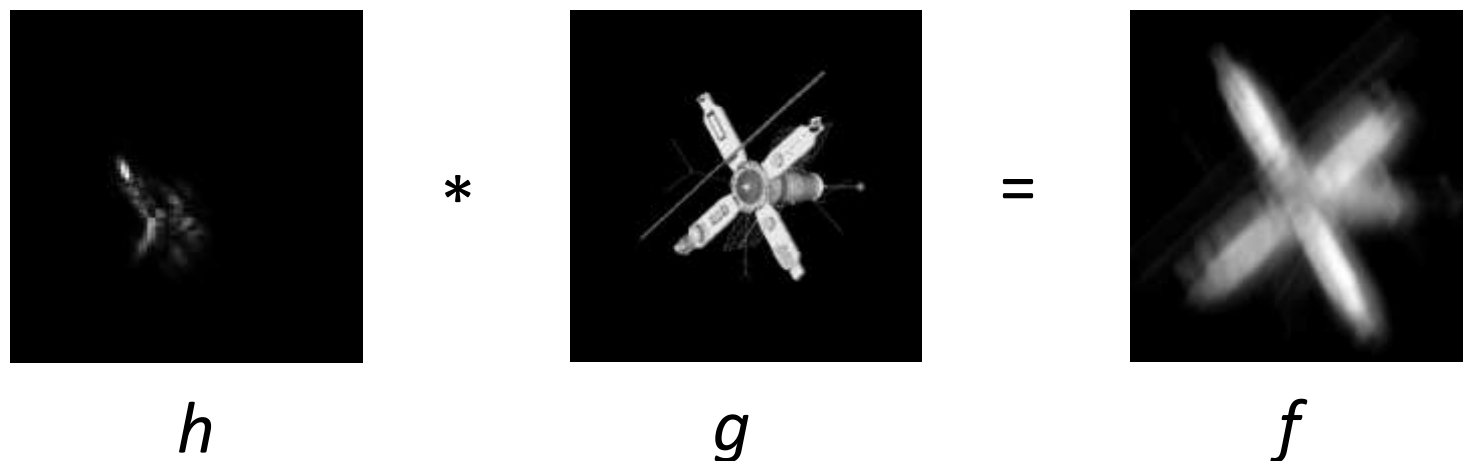
Trilinear Interpolation

- **Trilinear interpolation** is a straightforward generalization of the bilinear case to hexahedral cells in 3D
 - Three local coordinates α, β, γ
 - Again, does not result in a linear function
 - Piecewise bi-/trilinear interpolation is again only C^0 continuous between cells



Convolution: Intuition

- **Convolution** is an operation on two given functions h, g that produces another function f
 - *Intuition*: f obtained as weighted sum of shifted versions of g . The weights of different shifts are given by h
 - *Example*: Photograph with motion blur
 - In practice, “kernel” h often has much smaller support than g
 - Note that h and g are interchangeable, since convolution is *commutative*



Continuous Convolution

- **Definition:** Convolution of continuous functions g and h :

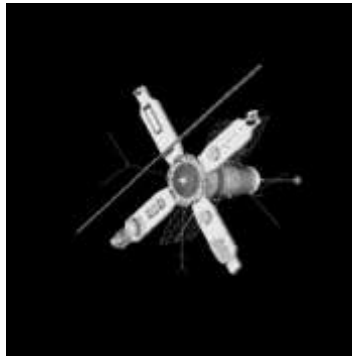
$$f(x) = (h * g)(x) = \int_{-\infty}^{\infty} h(\xi) \cdot g(x - \xi) d\xi$$

- Intuition: Negative ξ should shift g to the left
- Minus sign makes convolution associative and commutative

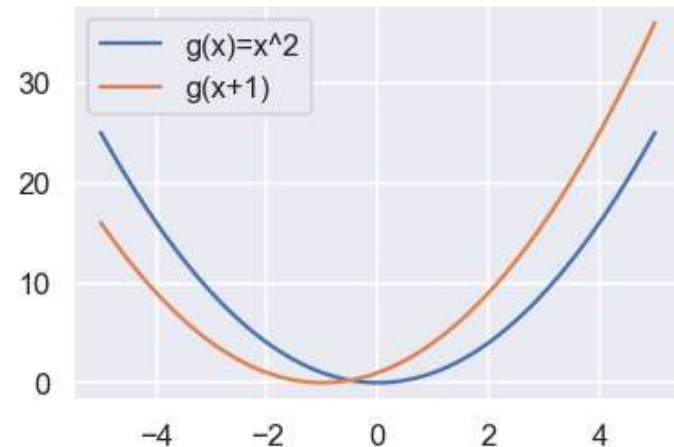


h

*



g



Separable Convolution in 2D/3D

- Filtering/reconstructing 2D images requires 2D convolution:
 $\mathbf{x}/\xi$ are 2D vectors

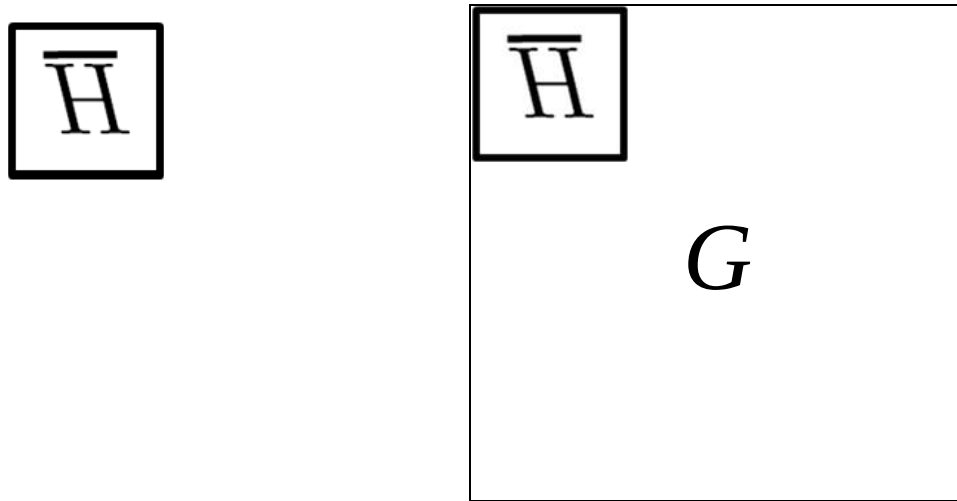
$$(h * g)(\mathbf{x}) = \iint h(\xi)g(\mathbf{x} - \xi)d\xi$$

- The use of **separable kernels** h simplifies this:

$$(h * g)(\mathbf{x}) = \iint h_1(\xi_1)h_2(\xi_2)g(x_1 - \xi_1, x_2 - \xi_2)d\xi$$

- We often use $h_1=h_2$
- Generalization to 3D straightforward

Illustration: Convolution



Dirac Delta

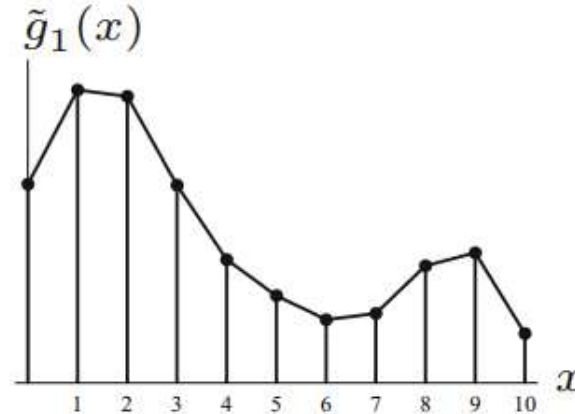
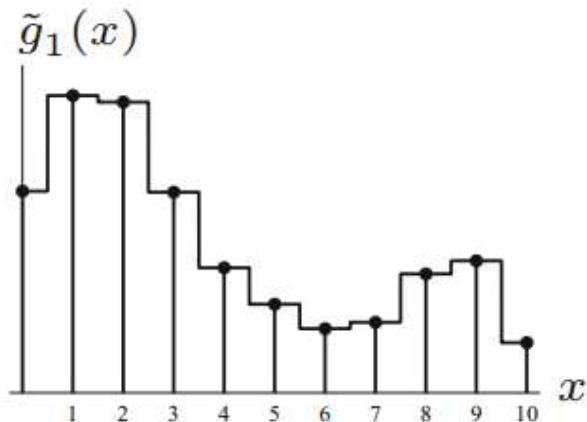
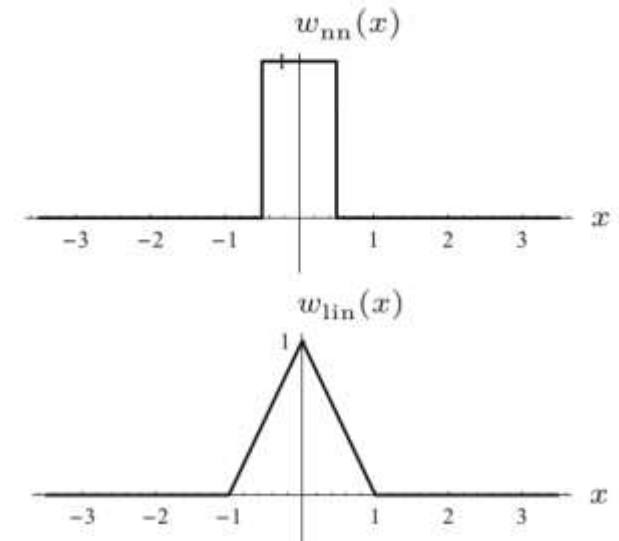
- The Dirac delta $\delta(x)$
 - is normalized: $\int_{-\infty}^{\infty} \delta(x) dx = 1$
 - can be used to evaluate a function $f(x)$ at a by multiplication and integration over x :

$$\int_{-\infty}^{\infty} f(x) \delta(x - a) dx = f(a)$$

- Informally, $\delta(x)$ is often thought of as a function that is $\delta(x) = 0$ for all x except $\delta(0) = \infty$.
 - Graphical depiction of $\delta(x - a)$: Vertical bar at $x = a$
 - We will represent value y at discrete position a as $y \delta(x - a)$

Convolution-Based Interpolation

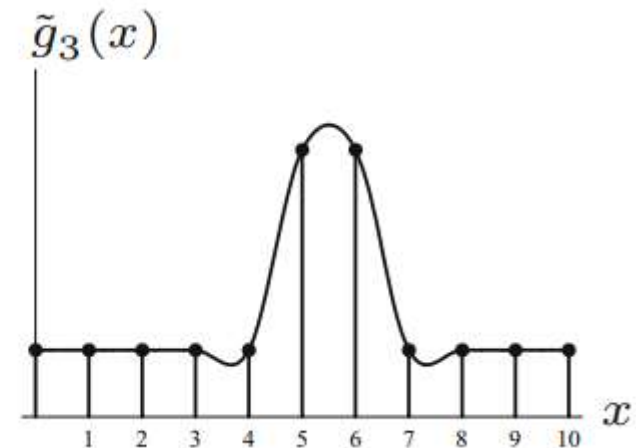
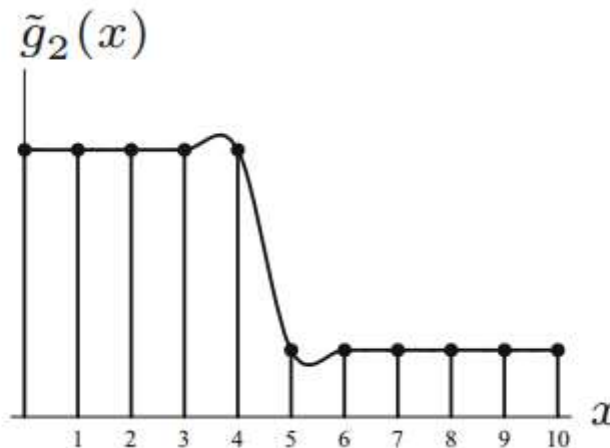
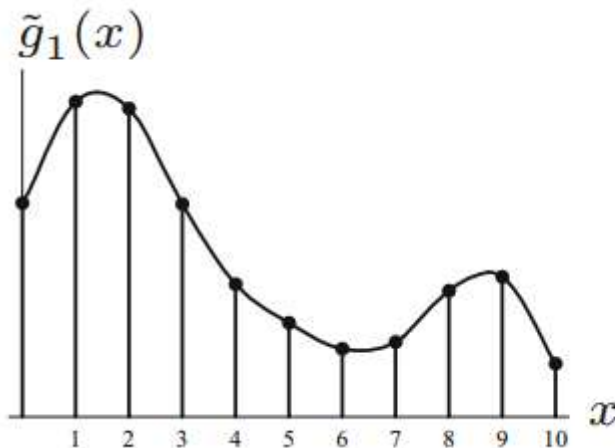
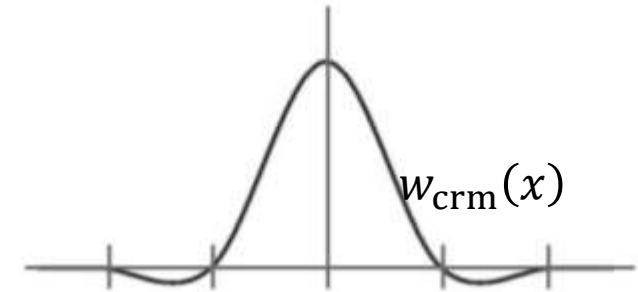
- Many interpolation schemes can be expressed as convolving a discrete signal (x_i, y_i) , $i = 1, \dots, n$, described by a weighted sum of Dirac peaks $g(x) = \sum_{i=1}^n y_i \delta(x - x_i)$, with a continuous interpolation kernel $w(x)$
 - Nearest neighbor / linear interpolation can be expressed as convolution with a box / tent kernel
 - Bi-/trilinear interpolation obtained via separable convolution



Cubic Spline Interpolation

- **Piecewise cubic polynomials** on $(-2,2)$ provide smoothness with limited computational effort (up to four samples)
- **Catmull-Rom spline** is particularly widely used:

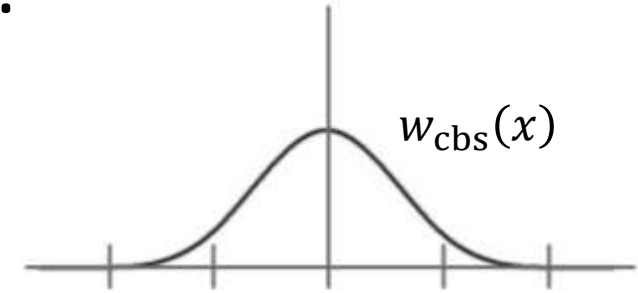
$$w_{\text{crm}}(x) = \frac{1}{2} \begin{cases} 3|x|^3 - 5|x|^2 + 2 & \text{for } 0 \leq |x| < 1 \\ -|x|^3 + 5|x|^2 - 8|x| + 4 & \text{for } 1 \leq |x| < 2 \\ 0 & \text{for } |x| \geq 2 \end{cases}$$



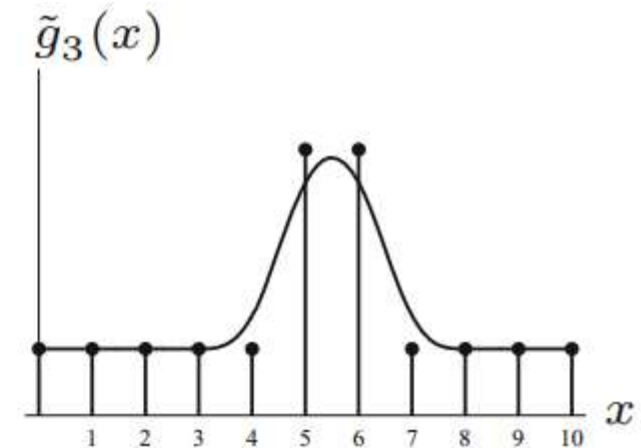
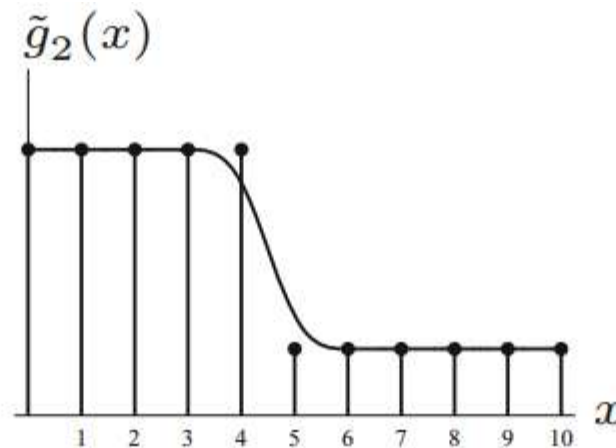
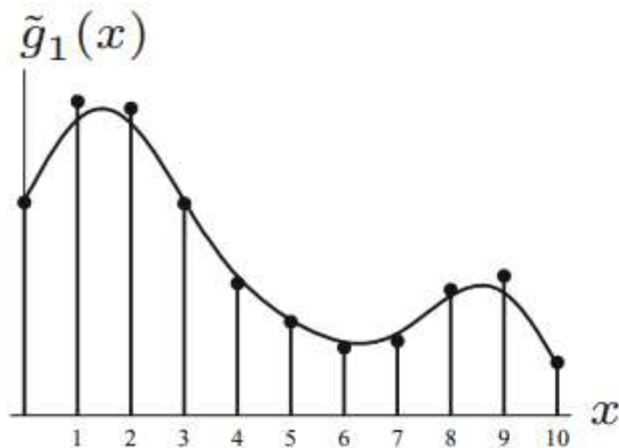
Cubic B-spline Approximation / Interpolation

- A C^2 continuous alternative is the **cubic B-spline**:

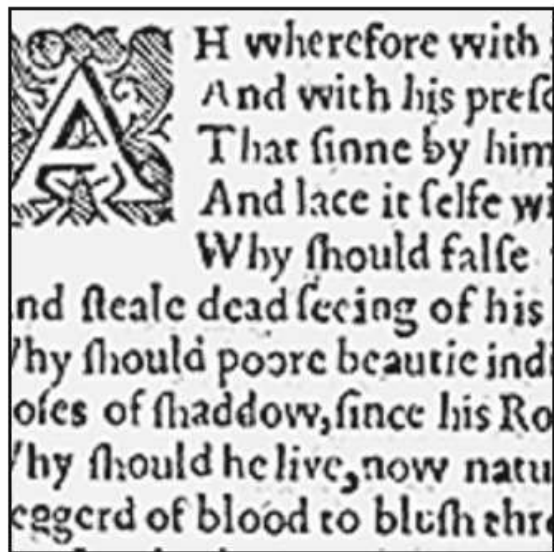
$$w_{cbs}(x) = \frac{1}{6} \begin{cases} 3|x|^3 - 6|x|^2 + 4 & \text{for } 0 \leq |x| < 1 \\ -|x|^3 + 6|x|^2 - 12|x| + 8 & \text{for } 1 \leq |x| < 2 \\ 0 & \text{for } |x| \geq 2 \end{cases}$$



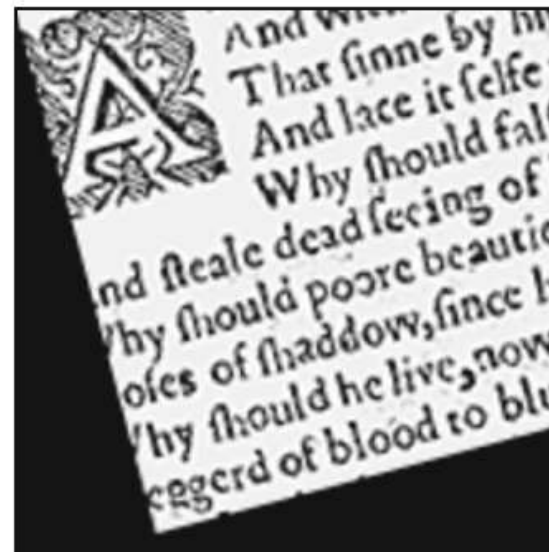
- No oscillations, no over- or undershoots
- **Approximates**; use for interpolation requires suitable pre-filtering



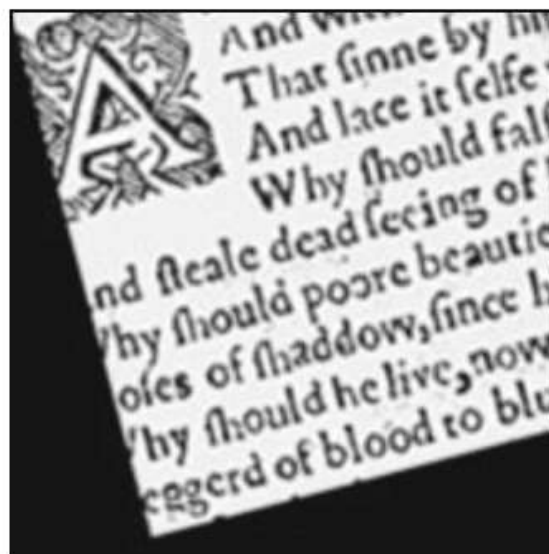
Example: Image Rotation with Different Kernels



Nearest Neighbor
Catmull-Rom



Bilinear



Cubic B-spline
Approximation

Boundary Conditions

- **Problem:** At the boundary, the support of the convolution (or cross-correlation) kernel extends beyond the image
- **Solutions:**
 - Ignore / cut off boundary (“valid” part of the convolution)
 - Padding
 - With a fixed value (typically zero)
 - By repeating the value at the boundary
 - By mirroring values at across the boundary
 - By periodically repeating the image

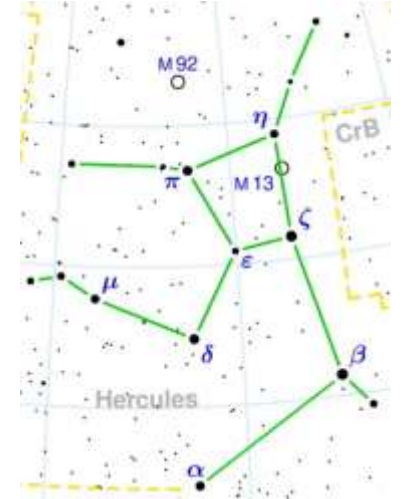
Summary: Interpolation

- **Linear interpolation** is popular due to its simplicity and speed
 - **Barycentric interpolation** on simplices
 - **Bi-/trilinear interpolation** on rectangles / hexahedra
- **Convolution-based interpolation** is the common way to get smooth reconstructions
 - **Kernel** determines smoothness, computational effort, oscillations
 - **Cubic spline interpolation** is a popular trade-off between smoothness and computational cost

8.5 Transforming Coordinate Systems

Illustrating Example

- Arecibo Message:
 - Arecibo observatory, located on Puerto Rico
 - Built in 1964, renovated in 1974
 - To celebrate, a message was sent to star cluster M13 in Hercules
- Historical footnotes:
 - Until July 2016, largest single-aperture radio telescope on earth (305m)
 - Collapsed in Dec 2020



Arecibo Message

- Message sent at 2380 MHz
- 1679 bits encoded by shifting 10 Hz:

[illegible]

- As an alien, how do you make sense of this?

Axis Order

- Aliens may notice that $1679 = 23 \times 73$ (product of two primes) and decide to try a 2D layout.
- But should a new scanline start after 23 or after 73 pixels?



23x73



73x23

https://en.wikipedia.org/wiki/Arecibo_message to find out what the message means...

Implementing Uniform Grids

Computer memory is addressed linearly, so we need to store value at 3D voxel coordinate (i,j,k) at a memory location a

- One option, assuming $n_x \times n_y \times n_z$ voxels:

$$a = k + n_z * (j + n_y * i)$$

Example in Python/numpy:

```
import numpy as np
nx=5; ny=7; nz=4
i=3; j=5; k=2
voxeldata = np.zeros((nx,ny,nz))
voxeldata[i,j,k]=1
a=voxeldata.ravel()
print(a[k+nz*(j+ny*i)])
```

Output: 1.0

Fast vs. Slow Axes

From previous slide: $a = k + nz * (j + ny * i)$

Computing the inverse mapping in Python:

```
i = a // (ny * nz)
```

```
j = (a // nz) % ny
```

```
k = a % nz
```

```
// = integer division  
%  = modulo operator
```

As we step through computer memory:

- the values of k change most frequently
→ “fastest axis”
- the values of i change least frequently
→ “slowest axis”

Fast vs. Slow Axes in 2D (Matrix) Case

An m -by- n matrix (m rows, n columns) $M_{i,j}$ with row index i and column index j is said to be in

“row major” order if
 j is faster than i

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

used in Python and C:
`double M[m][n];`

“column major” order if
 i is faster than j

$$\begin{pmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{pmatrix}$$

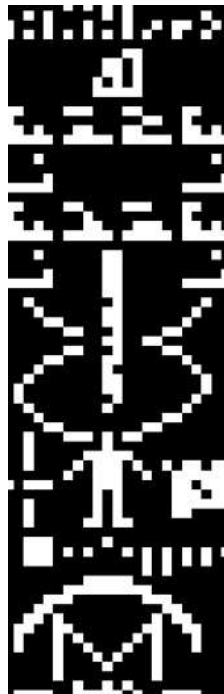
used in MATLAB,
Fortran, GLSL, ...



Pitfall when using numerical packages!
Axis order important part of meta data!

Index to World Space

- Even once the dimension of the fast axis has been identified, several options remain to spatially lay out pixel positions:



vs.



vs.



...

Index Coordinates vs. World Coordinates

The voxel coordinates (i,j,k) from which we compute the index in memory are also called “index coordinates”.

The 3D “world coordinates” (x,y,z) at which the sample was measured or simulated are computed from (i,j,k) using an origin $\mathbf{p}$ and three vectors $(\mathbf{e},\mathbf{f},\mathbf{g})$ that correspond to a step along the respective array axis:

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \mathbf{p} + i\mathbf{e} + j\mathbf{f} + k\mathbf{g}$$

Linear Transformations

- Consider functions $\mathbf{y}=T(\mathbf{x})$ that map one vector $\mathbf{x}$ to another vector $\mathbf{y}$.
- T is a **linear transformation** if
 - $T(\mathbf{v}+\mathbf{w})=T(\mathbf{v})+T(\mathbf{w})$
 - $T(\alpha\mathbf{v})= \alpha T(\mathbf{v})$
 - It can be written as a matrix-vector multiplication: $T(\mathbf{x})=\mathbf{T}\mathbf{x}$
- Most fundamental geometric transformations (scaling, rotation etc.) are linear

Scaling

- **Uniform scaling** by factor α : $\mathbf{S} = \alpha \mathbf{I}$



- **Non-uniform scaling** along a unit vector $\hat{\mathbf{x}}$:

– Based on decomposition via $\mathbf{P}_{\hat{\mathbf{x}}} = \hat{\mathbf{x}}\hat{\mathbf{x}}^T$ and $\mathbf{P}_{\hat{\mathbf{x}}}^{\perp} = \mathbf{I} - \hat{\mathbf{x}}\hat{\mathbf{x}}^T$

$$\mathbf{v} = \mathbf{I}\mathbf{v} = \mathbf{P}_{\hat{\mathbf{x}}}\mathbf{v} + \mathbf{P}_{\hat{\mathbf{x}}}^{\perp}\mathbf{v}$$

– Resulting equation for scaling by factor α along $\hat{\mathbf{x}}$:

$$\mathbf{S}_{\alpha, \hat{\mathbf{x}}} = \alpha \mathbf{P}_{\hat{\mathbf{x}}} + \mathbf{P}_{\hat{\mathbf{x}}}^{\perp}$$

• Uniform scaling by factor α : $\mathbf{S} = \alpha \mathbf{I}$

• Non-uniform scaling along a unit vector $\hat{\mathbf{x}}$:

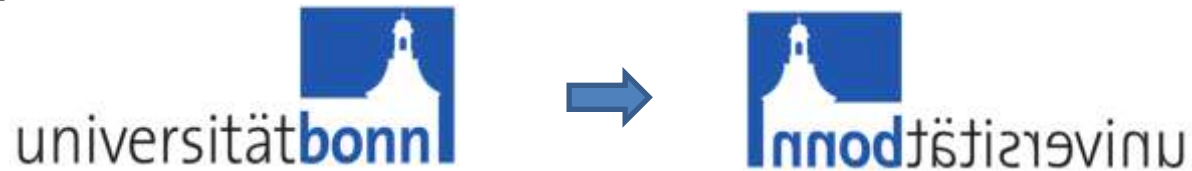
– Based on decomposition via $\mathbf{P}_{\hat{\mathbf{x}}} = \hat{\mathbf{x}}\hat{\mathbf{x}}^T$ and $\mathbf{P}_{\hat{\mathbf{x}}}^{\perp} = \mathbf{I} - \hat{\mathbf{x}}\hat{\mathbf{x}}^T$
 $\mathbf{v} = \mathbf{I}\mathbf{v} = \mathbf{P}_{\hat{\mathbf{x}}}\mathbf{v} + \mathbf{P}_{\hat{\mathbf{x}}}^{\perp}\mathbf{v}$



Reflection and Shear

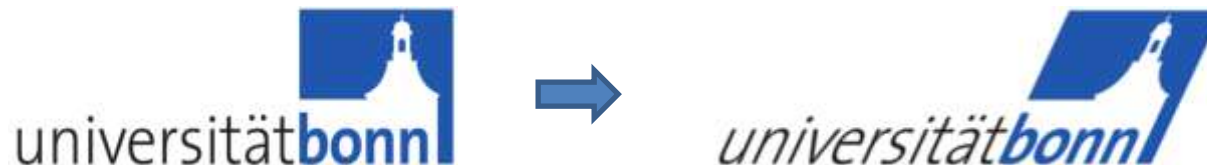
- **Reflection** special case of non-uniform scaling with $\alpha = -1$:

$$\mathbf{M}_{\hat{\mathbf{x}}} = \mathbf{P}_{\hat{\mathbf{x}}}^{\perp} - \mathbf{P}_{\hat{\mathbf{x}}}$$



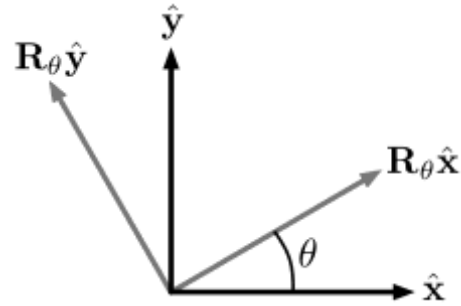
- **Shear** (transvection): Parallel lines are shifted by an amount that is proportional (with shearing factor β) to the (signed) distance along a perpendicular vector

$$\mathbf{T} = \mathbf{I} + \beta \hat{\mathbf{x}} \hat{\mathbf{y}}^T$$



Rotation Matrix

- Within plane $\langle \{\hat{\mathbf{x}}, \hat{\mathbf{y}}\} \rangle$, rotation of $\hat{\mathbf{x}}$ towards $\hat{\mathbf{y}}$ by angle θ is achieved by the following rotation matrix:



$$\mathbf{R}_\theta \hat{\mathbf{x}} = \cos \theta \hat{\mathbf{x}} + \sin \theta \hat{\mathbf{y}}$$

$$\mathbf{R}_\theta \hat{\mathbf{y}} = -\sin \theta \hat{\mathbf{x}} + \cos \theta \hat{\mathbf{y}}$$

$$\mathbf{R}_\theta \mathbf{a} = \mathbf{a}.$$

$$\mathbf{R}_\theta = \mathbf{I} + (\cos \theta - 1)(\hat{\mathbf{x}}\hat{\mathbf{x}}^T + \hat{\mathbf{y}}\hat{\mathbf{y}}^T) + \sin \theta (\hat{\mathbf{y}}\hat{\mathbf{x}}^T - \hat{\mathbf{x}}\hat{\mathbf{y}}^T)$$

- Example with $\hat{\mathbf{x}} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$, $\hat{\mathbf{y}} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$: $\mathbf{R}_\theta = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$



Translations and Affine Transformations

- **Translation** $f(x)=x+t$ is a fundamental operation, but cannot be represented as a matrix-vector product in the original space
 - Requires moving from linear to **affine** maps: $f(x)=Mx+t$
- Solved by **homogeneous coordinates**:
 - Increase the dimension of x by using 1 as the additional coefficient
 - Translation becomes part of the **affine transformation matrix**:

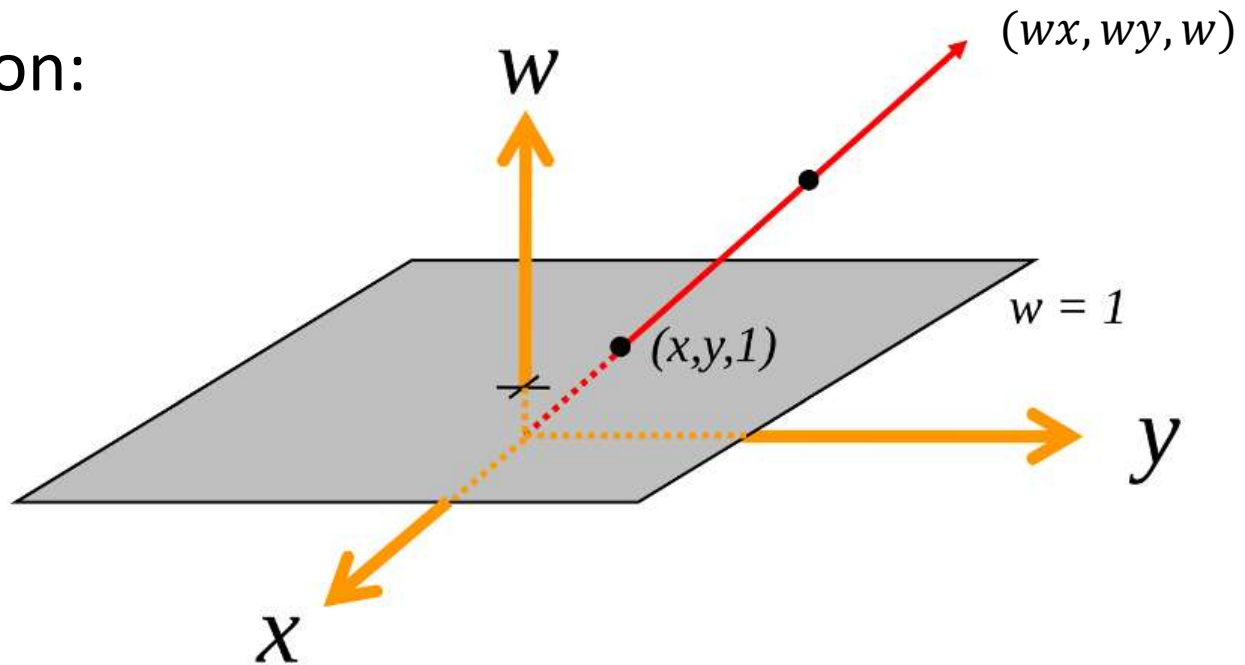
$$\begin{pmatrix} a & b & c & t_x \\ d & e & f & t_y \\ g & h & i & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Scaling +
Rotation +
Shear

Translation

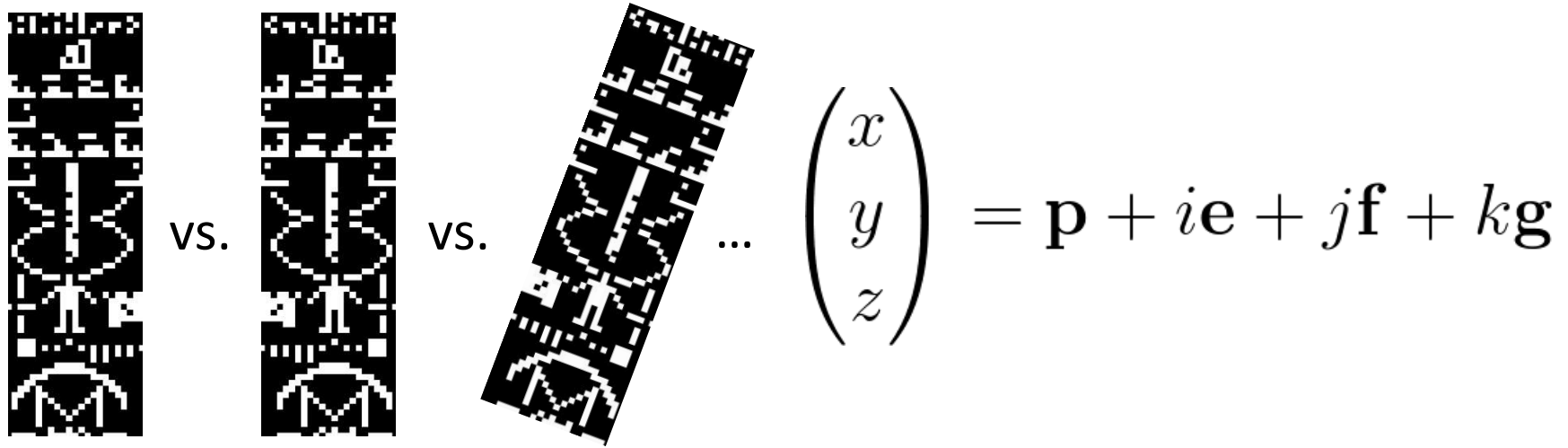
Homogeneous Coordinates

- In **homogeneous coordinates**, n -D points are represented by lines in a $(n+1)$ -D space:
 - Additional coordinate w
 - (x,y) represented as (wx,wy,w)
 - Canonical representation:
 $(x,y,1)$
 - Vectors represented with $w=0$



Example: Index to World Mapping

Remember mapping index to world space:



In homogeneous coordinates, that is written as:

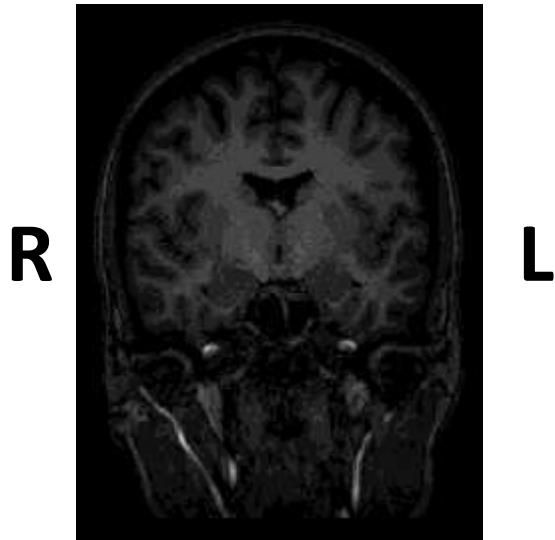
$$\begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} | & | & | & | \\ \mathbf{e} & \mathbf{f} & \mathbf{g} & \mathbf{p} \\ | & | & | & | \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} i \\ j \\ k \\ 1 \end{pmatrix}$$

Handedness of the Coordinate System

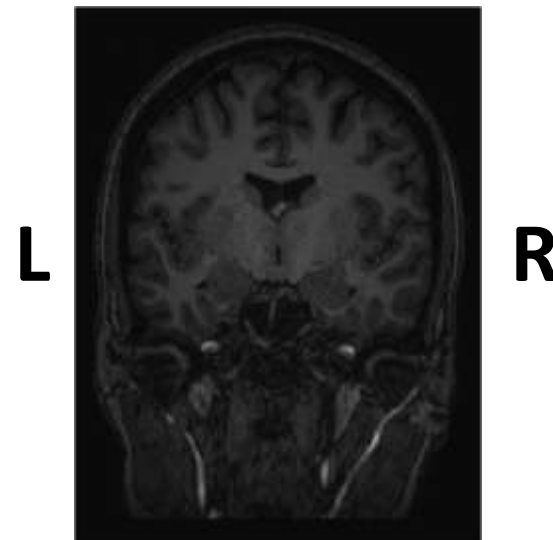
Specifying coefficients of $(\mathbf{e}, \mathbf{f}, \mathbf{g})$ assumes a canonical coordinate system, which can be either right- or left-handed.



Contradictory conventions sometimes exist in the same field (e.g., neuroimaging)!



Radiological



Neurological

Summary: Transforming Coordinate Systems

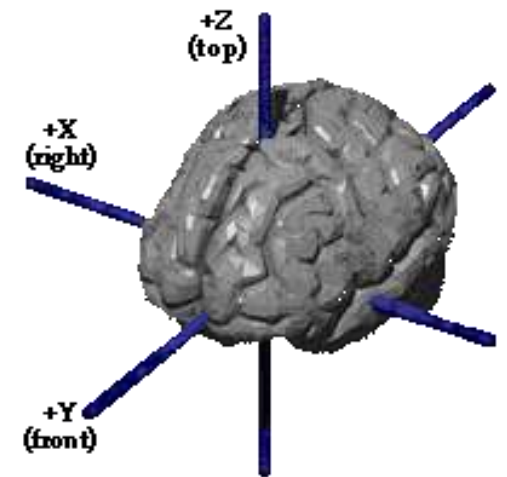
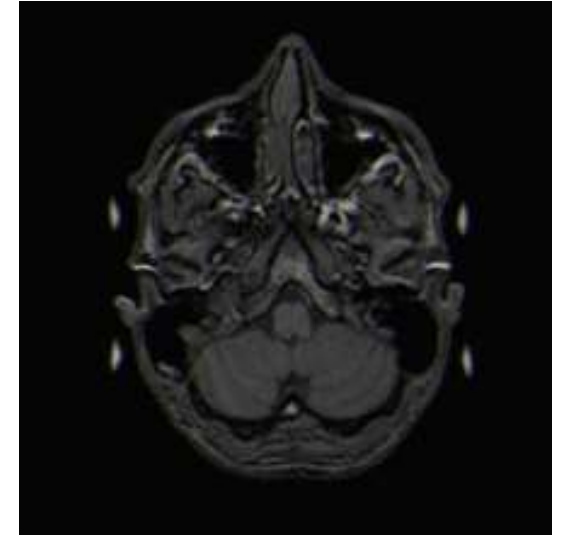
- **Index space** tells us where to find a sample in memory
- **World space** tells us where it was measured and where it should be visualized
 - Specification requires origin, axis vectors, handedness of the coordinate system
- We can convert between the two using simple **matrix-vector multiplication**
 - Requires **homogeneous coordinates** to represent translations via matrices
 - Reminder of how common **geometric transformations** can be represented as matrices

8.6 Slicing

Slicing

Map data on a series of 2D planes through the volume

- In radiology, this is still the most frequently used visualization
- If data was acquired as a series of 2D slices, natural to show those
- Other axis-aligned views
- Rotate (“re-angulate”) to an object/patient-aligned view
 - Involves interpolation (Section 8.4)



Slicing: Implementation

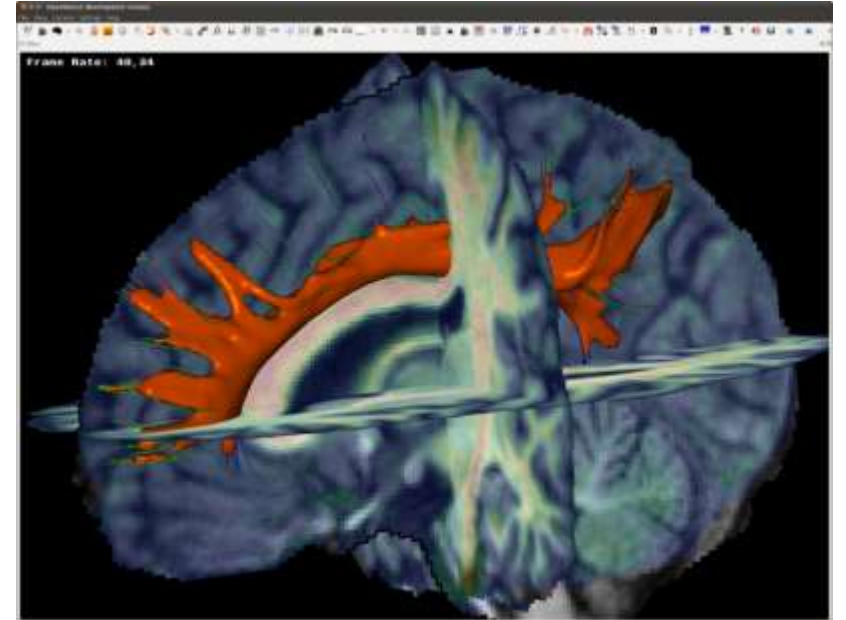
Basic idea: Render textured slice geometry (e.g., simple quads)

Most convenient to use **3D textures**:

- If desired, (trilinear) interpolation is performed by the graphics hardware
- For *pre-shading*, map volume to color / grayscale and upload RGB texture
- For *post-shading*, upload volume itself and map the interpolated value in a fragment shader, e.g., via windowing parameters or a 1D texture lookup

Slicing: Implementation Details

1. Index coordinates for the 3D texture lookup can be found from world-space (geometric) coordinates via the inverse of the index-to-world matrix
 - Section 8.5
2. If desired, shader programming makes it easy to discard fragments that correspond to air

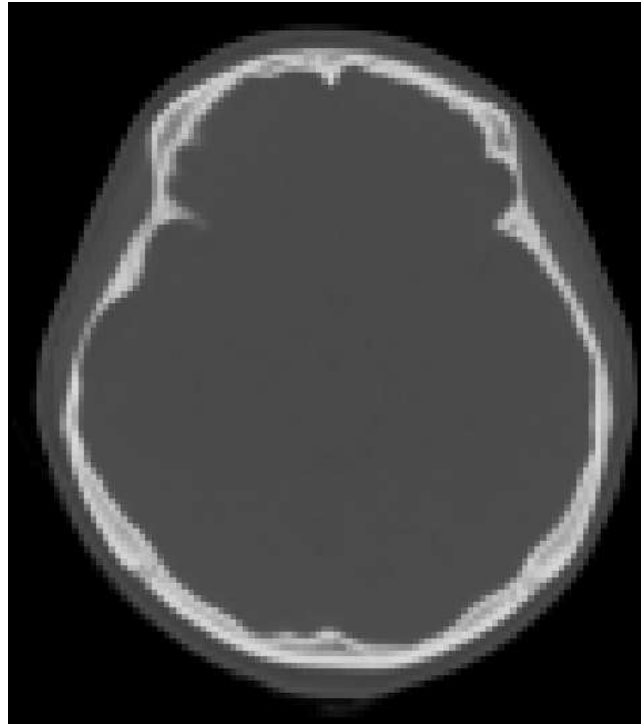


Windowing

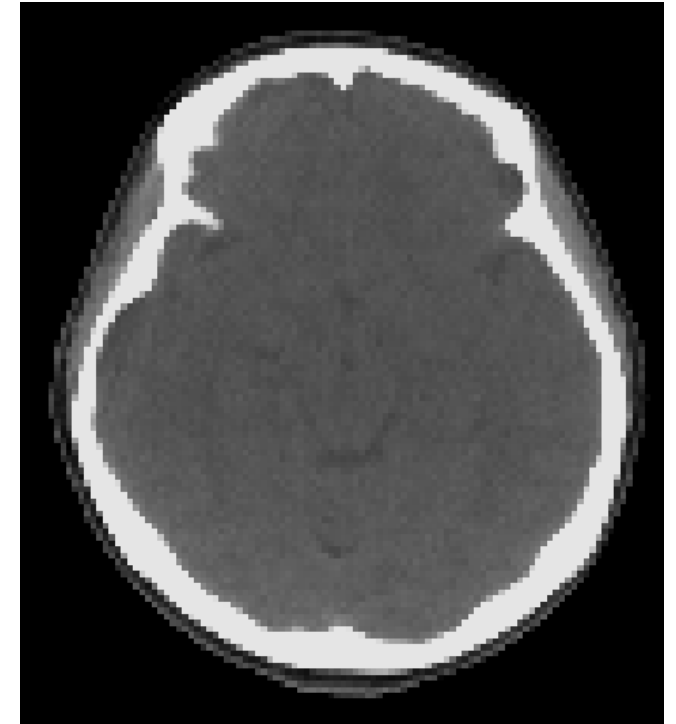
For slicing, we often map a range of data values (**“window”**) to a linear ramp of grayscales. Values outside that range are mapped to black or white, respectively.



Brain Window



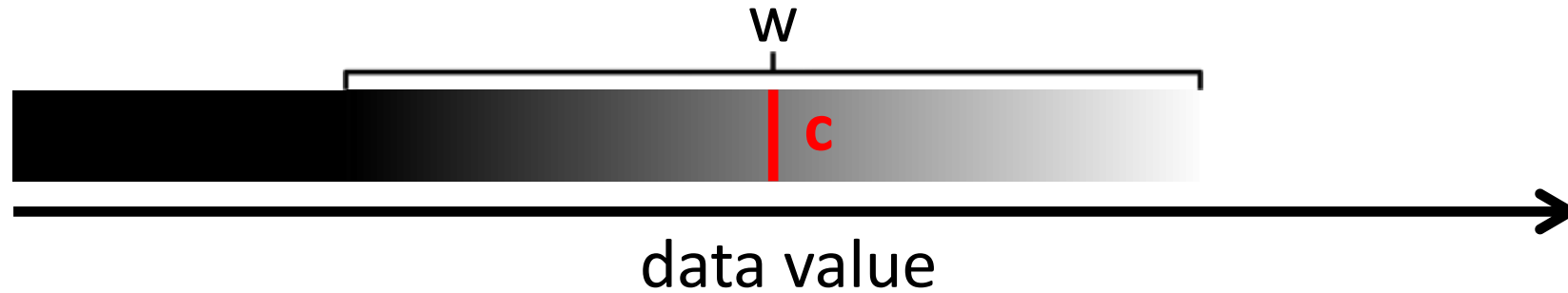
Bone Window



Subdural Window

Window Specification

Windows are commonly defined by their center c and width $w \geq 1$.



For data value x and color output range $C \in [0,1]$, DICOM specifies the following pseudo-code:

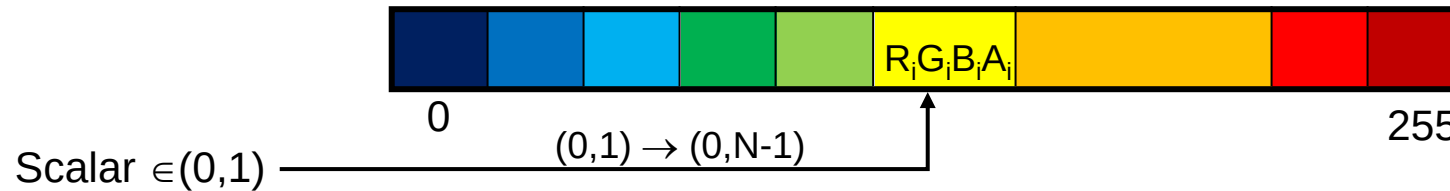
```
if (x <= c - 0.5 - (w-1)/2) then C := 0
else if (x > c - 0.5 + (w-1)/2) then C := 1
else C := (x - (c - 0.5)) / (w-1) + 0.5
```

- Division is floating point without truncation.
- $w=1$ performs binary thresholding

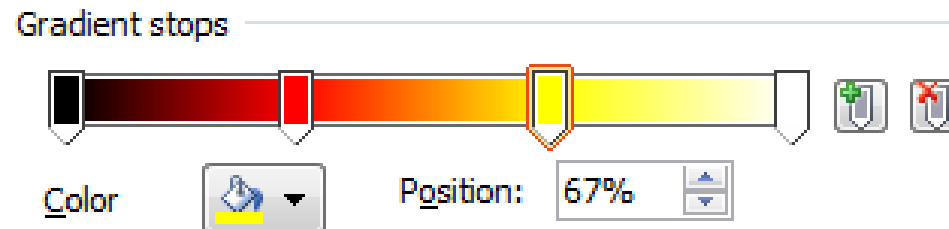
Color Maps

Alternatively, can map scalar data to color

- Can be implemented as a color lookup table:

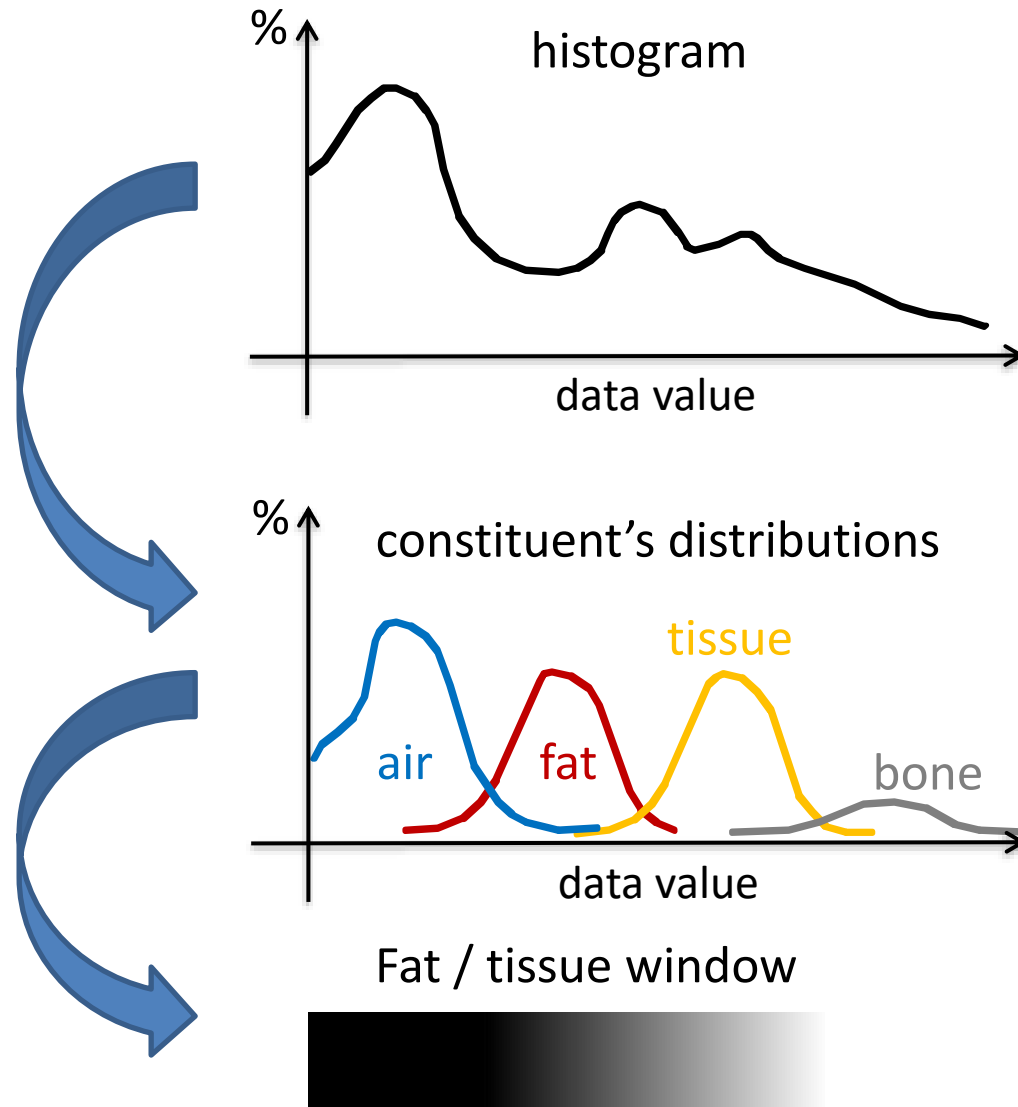


- Often specified by defining the color for a discrete set of data values (“gradient stops”) and interpolating in between
 - *Quiz:* Good / convenient ways of interpolation?



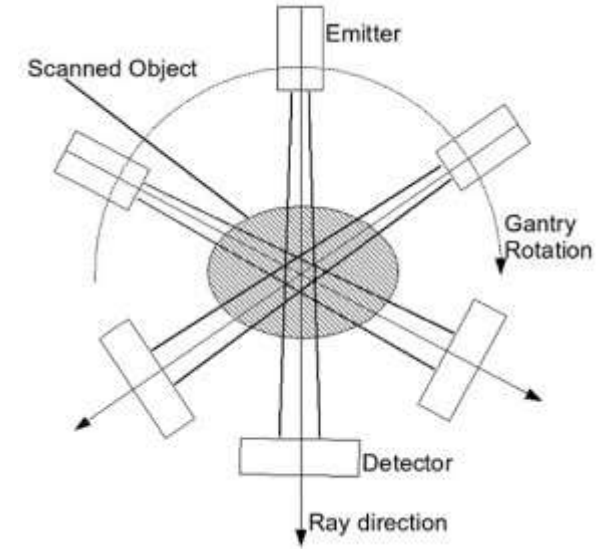
Histograms Can Guide Windowing / Color Mapping

Histogram of all data values in a volume dataset:



Computed Tomography (CT/CAT)

- Introduced in 1968, Nobel Prize in 1979
- Based on measuring X-ray absorption from many different directions
 - Algorithmic reconstruction of a slice image
 - Multiple slices combined into a 3D volume
 - Often acquired simultaneously
- Can achieve sub-mm voxel size



Hounsfield Units

Hounsfield units (HU) for CT data sets

- Sir Godfrey Newbold Hounsfield (1919-2004)
- Quantifies X-ray density of the material
- Calibrated to attenuation coefficient of water (HU=0)

$$\text{HU} := 1000 \times \frac{\mu_X - \mu_{\text{water}}}{\mu_{\text{water}}}$$

- In CT, discretized in range -1024 to +3071 HU
 - 12 bit CT-measurements

Hounsfield Units: Some Typical Values

Material	HU range
<i>Air</i>	<i>-1000</i>
Lung	≈ -600 to -400
Fat	≈ -100 to -20
<i>Water</i>	<i>0</i>
Muscle / Soft tissue	$\approx +20$ to $+80$
Bone	$> +500$

Note: HU for air and water are calibrated by definition. Exact values for other materials can depend on the scanner.

Summary: Data

- **Data classification** is based on its domain and values (scalar, vector, tensor)
- Common **data structures** include structured and unstructured grids
- Visualizing continuous phenomena based on discrete data involves **interpolation**
- Correct visualization requires transformation from **index** to **world coordinates**
- **Slicing** is the most fundamental visualization approach for volume data

Further Reading

- A.C. Telea. **Data Visualization: Principles and Practice**, A. K. Peters, 2007
- B. Preim, C. Botha, **Visual Computing for Medicine: Theory, Algorithms, and Applications**, Morgan Kaufmann, 2014
- Gerald Farin and Dianne Hansford. **Mathematical Principles for Scientific Computing and Visualization**. A. K. Peters, 2008
- Gordon L. Kindlmann. **Foundations of Scientific Visualization**, University of Chicago, 2013

Exam-like Questions

1. Suppose you would like to simulate and visualize the blood flow in the human heart. What type of grid would you prefer for this task?
2. Cubic spline interpolation provides a popular tradeoff between computational efficiency and smoothness. State two important differences between Catmull-Rom splines and cubic B-splines.
3. Write down an affine matrix for a nonuniform scaling by a factor of two along the y direction, followed by a translation along $[1,2,3]^T$. Will the matrix differ if we perform the nonuniform scaling after the translation?
4. What is the purpose of windowing in the context of slice-based visualizations?